9. Support Vector Machines

9.1 Maximal margin classifier

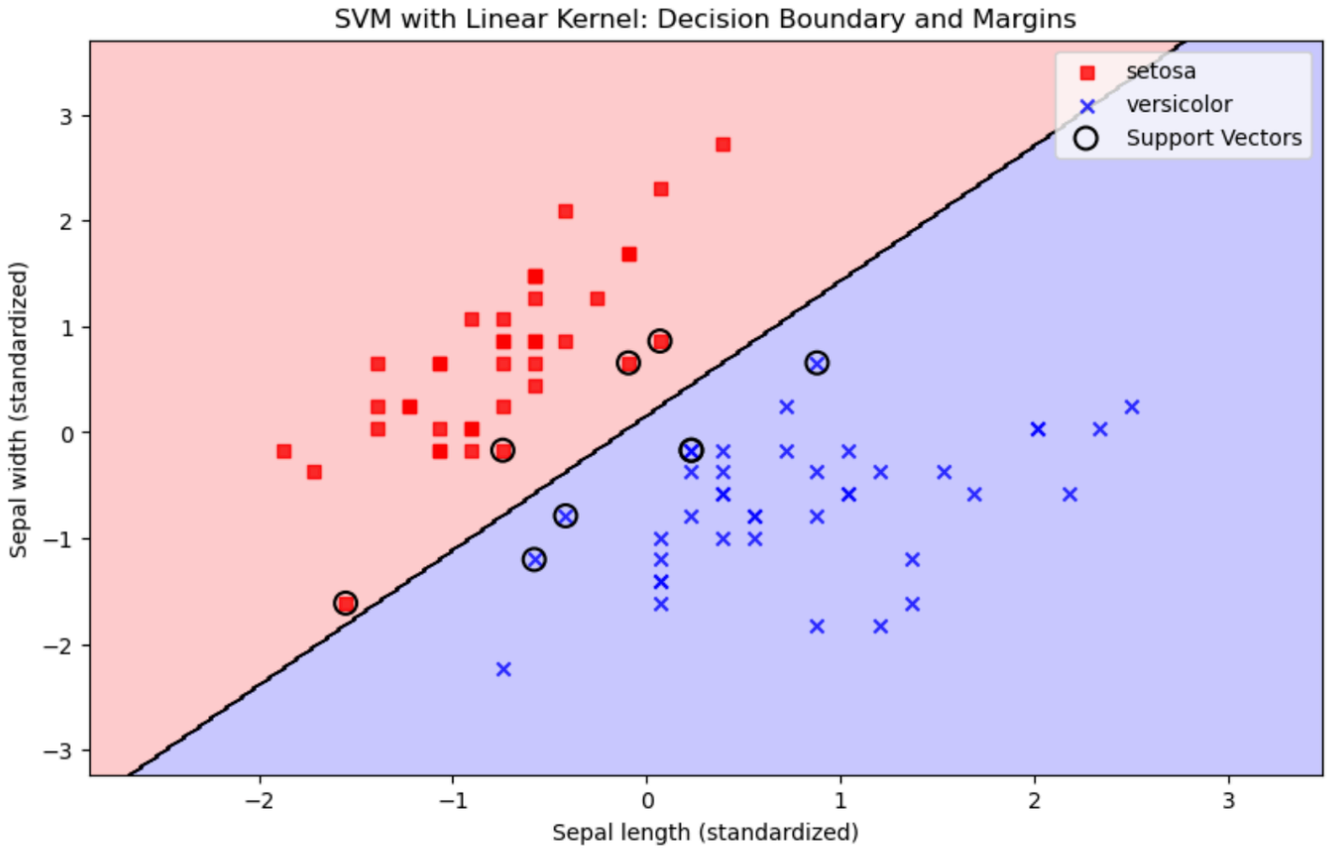
9.2 Support vector classifier

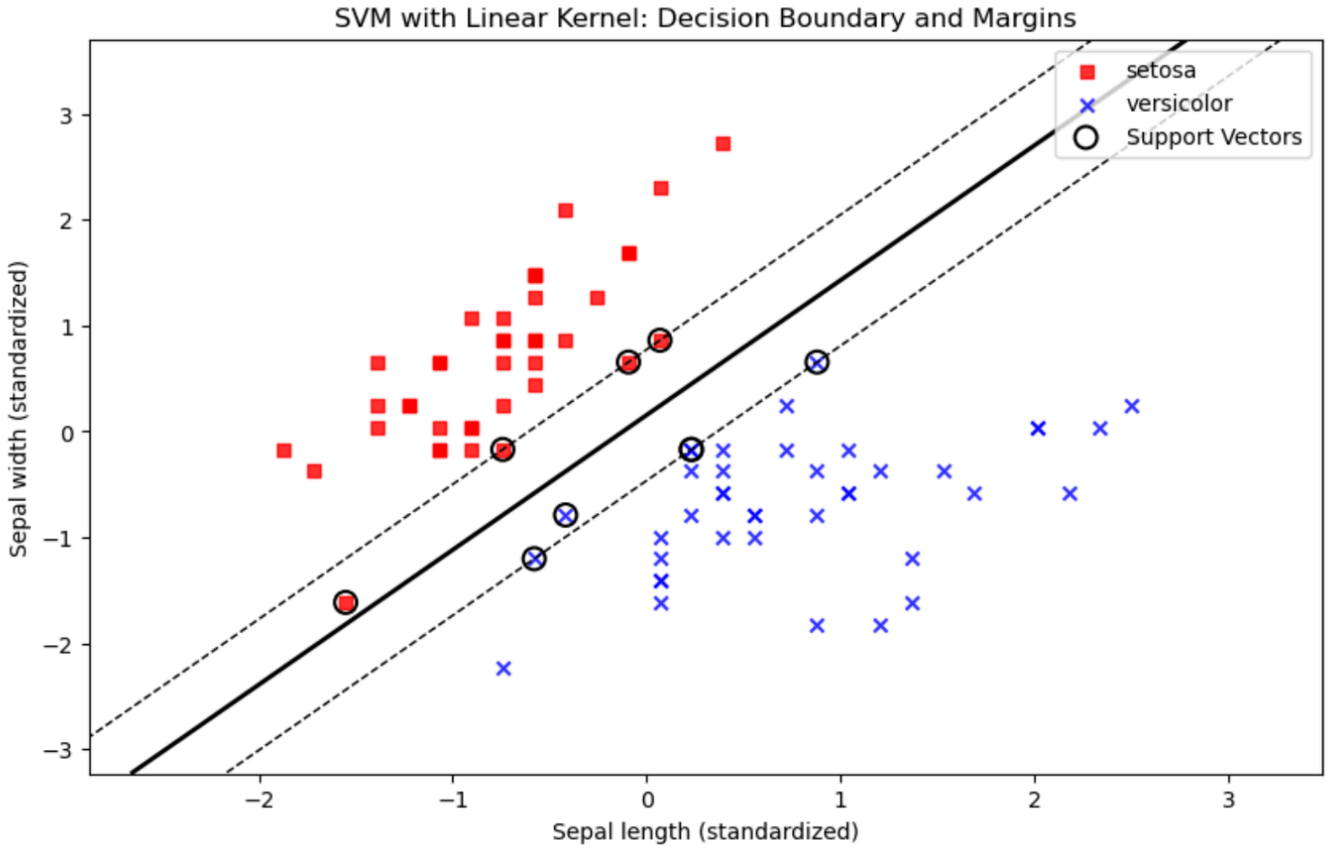
9.3 Support vector machine

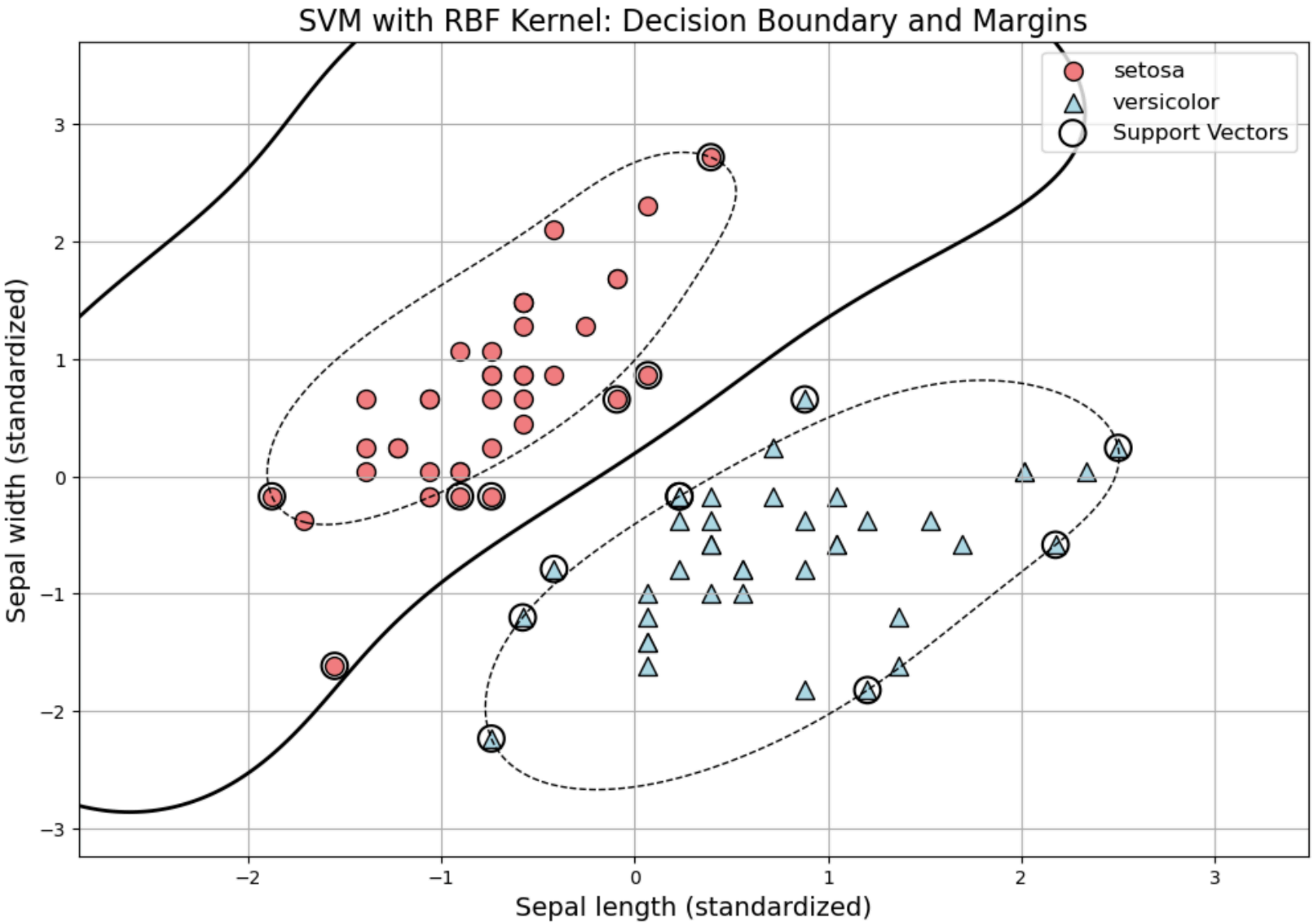
9.4 Primal-dual support vector classifiers

9.5 SVMs with more than two classes

9.6 Relationship to logistic regression


SVM with Linear Kernel: Decision Boundary and Margins
setosa
versicolor
Support Vectors
Sepal width (standardized)
Sepal length (standardized)
3
2
1
0
−1
−2
−3
−2
−1
0
1
2
3


SVM with Linear Kernel: Decision Boundary and Margins
setosa
versicolor
Support Vectors
Sepal width (standardized)
Sepal length (standardized)
3
2
1
0
−1
−2
−3
−2
−1
0
1
2
3


SVM with RBF Kernel: Decision Boundary and Margins
setosa
versicolor
Support Vectors
Sepal width (standardized)
Sepal length (standardized)
3
2
1
0
−1
−2
−3
−2
−1
0
1
2
3

9.0 Overview

About this chapter

- *Support vector machine is one of the most popular machine learning methodologies.*
- *Empirically successful, with well developed theory.*
- *One of the best off-the-shelf methods.*
- *We mainly address classification.*

9.1 Maximal Margin Classifier

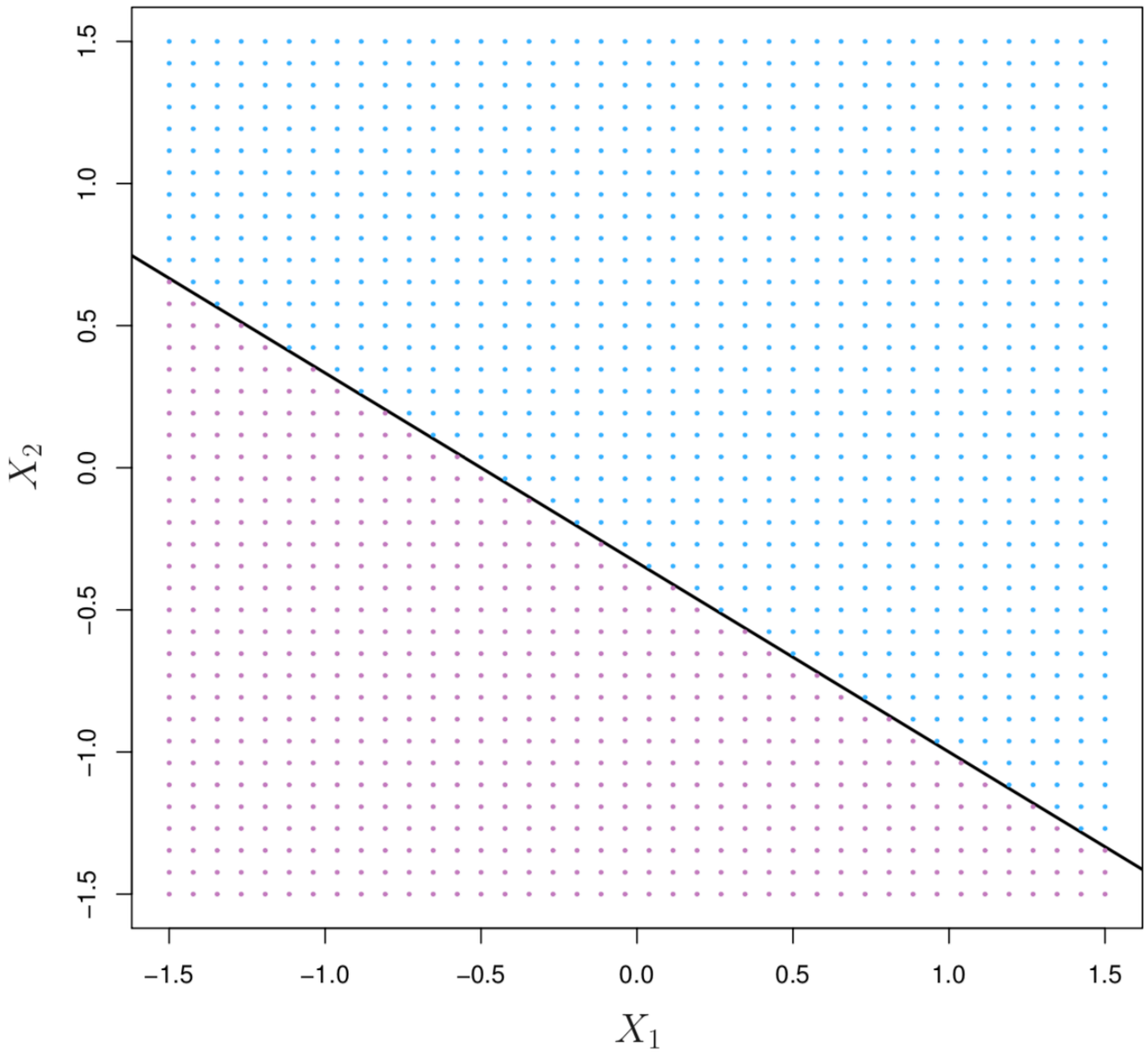


Figure: 9.1. The hyperplane $1 + 2X_1 + 3X_2 = 0$ is shown. The blue region is the set of points for which $1 + 2X_1 + 3X_2 > 0$, and the purple region is the set of points for which $1 + 2X_1 + 3X_2 < 0$.

9.1 Maximal Margin Classifier

Separating hyperplane

Training Data:$(y_i, \mathbf{x}_i), i = 1, ..., n$, with input $\mathbf{x}_i = (x_{i1}, ..., x_{ip})$ and two-class output $y_i = \pm 1$.

Suppose the two classes are seperated by one hyperplane

$$f(\mathbf{x}) = \beta_0 + \beta_1 x_1 + ... + \beta_p x_p,$$

meaning that, for all i in one class $y_i = 1$,

$$f(\mathbf{x}_i) > 0, \quad \text{one side of the hyperplane;}$$

and for all i in the other class $y_i = -1$,

$$f(\mathbf{x}_i) < 0, \quad \text{the other side of the hyperplane;}$$

It can be equivalently expressed as $y_i f(\mathbf{x}_i) > 0$, for all $i = 1, .., n$

9.1 Maximal Margin Classifier

Separating hyperplane

Prediction

If such separating hyperplane exists, it can be our classification rule:
For any new/old observation with $\mathbf{x}^*$ such that $f(\mathbf{x}^*) > 0$, classify it as in the class $+1$. Otherwise, classify it as in class -1.

***Issue**: if the two classes in training data are indeed separable by a hyperplane, which hyperplane is the best?*

9.1 Maximal Margin Classifier

Separating hyperplane

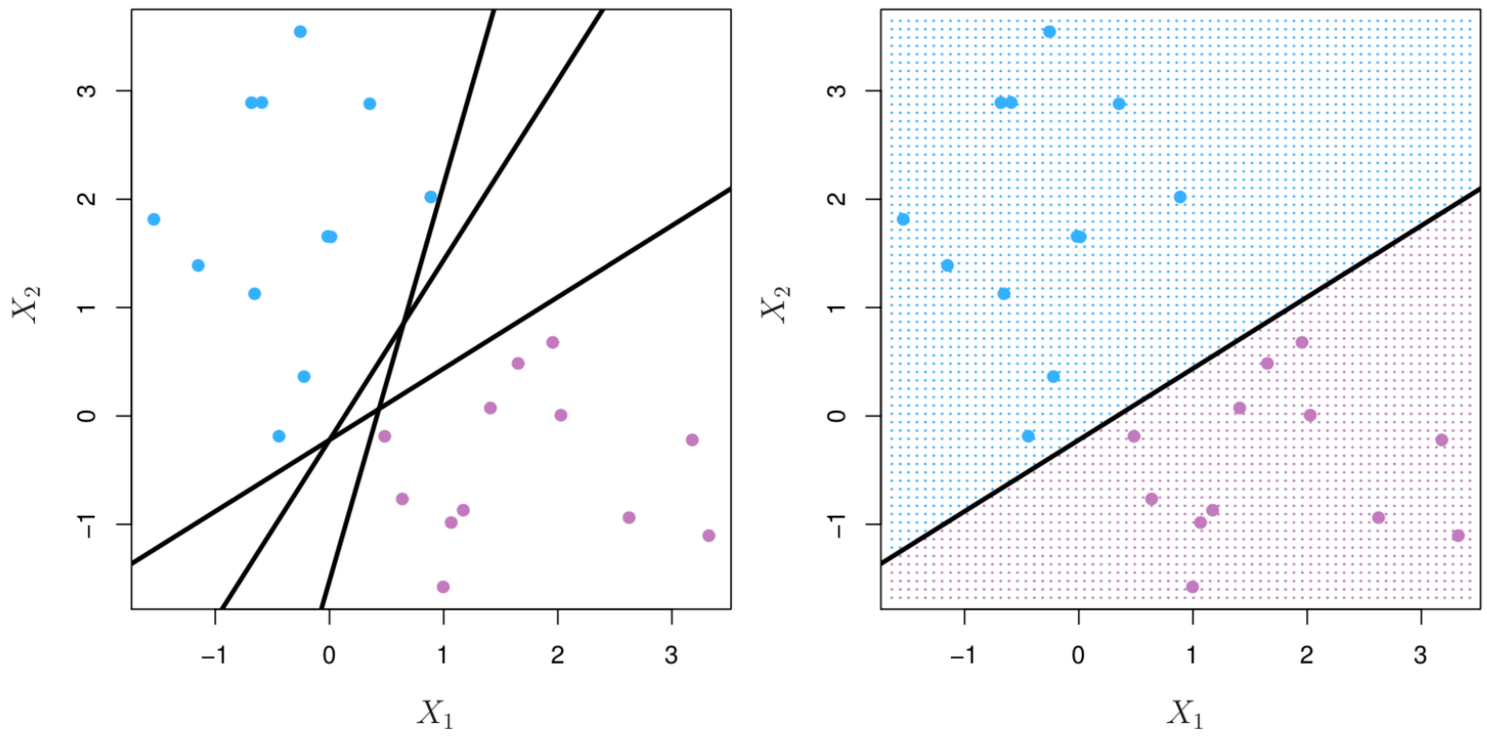


Figure: 9.2. Left: There are two classes of observations, shown in blue and in purple, each of which has measurements on two variables. Three separating hyperplanes, out of many possible, are shown in black. Right: A separating hyperplane is shown in black. The blue and purple grid indicates the decision rule made by a classifier based on this separating hyperplane: a test observation that falls in the blue portion of the grid will be assigned to the blue class, and a test observation that falls into the purple portion of the grid will be assigned to the purple class.

9.1 Maximal Margin Classifier

Maximal margin classifier

Maximal margin hyperplane

if the separating hyperplane that optimal separating hyperplane is farthest from the training observations.

The separating hyperplane such that the minimum distance of any training point to the hyperplane is the largest.

Creates a widest gap between the two classes.

Points on the boundary hyperplane, those with smallest distance to the max margin hyperplane, are called support vectors.
They "support" the maximal margin hyperplane in the sense vector that if these points were moved slightly then the maximal margin hyperplane would move as well .

9.1 Maximal Margin Classifier

Maximal margin classifier

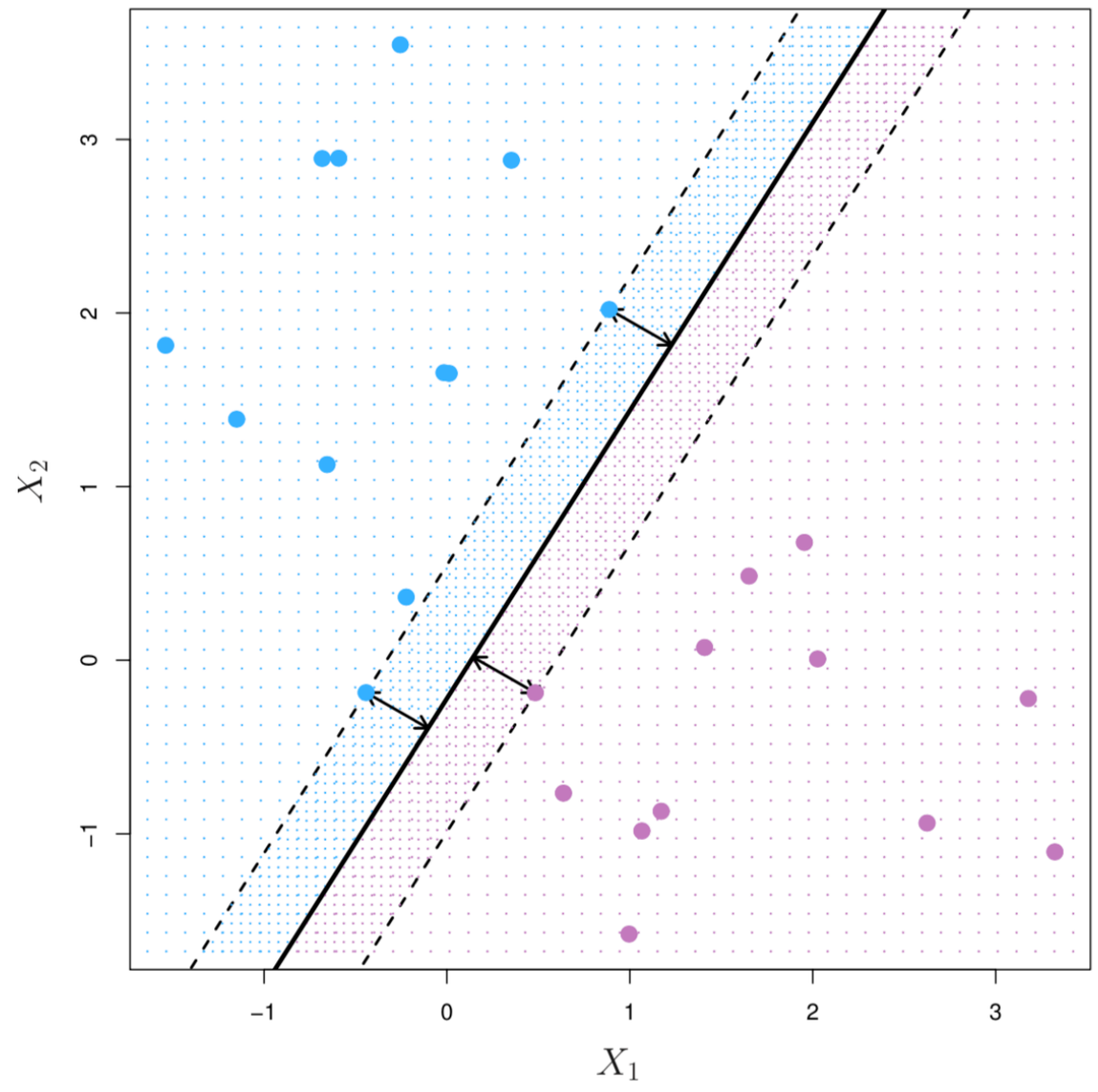


Figure 9.3 There are two classes of observations, shown in blue and in purple. The maximal margin hyperplane is shown as a solid line. The margin is the distance from the solid line to either of the dashed lines. The two blue points and the purple point that lie on the dashed lines are the support vectors, and the distance from those points to the hyperplane is indicated by arrows. The purple and blue grid indicates the decision rule made by a classifier based on this separating hyperplane.

9.1 Maximal Margin Classifier

Margin

Consider a data point $\mathbf{x}_0$. The distance from $\mathbf{x}_0$ to the hyperplane $\beta_0 + \beta^T \mathbf{x} = 0$ is given by

$$\min_{\mathbf{x}} \quad \|\mathbf{x}_0 - \mathbf{x}\|^2$$
$$\text{s.t. } \beta_0 + \beta^T \mathbf{x} = 0$$

The distance is then given by $\frac{|\beta_0 + \beta^T \mathbf{x}_0|}{\|\boldsymbol{\beta}\|}$.

For the seperable case, all data points can be correctly classified, i.e., $y_i(\beta_0 + \beta^T \mathbf{x}_i) > 0$ for all i.

Formulation of maximal margin

$$\arg\max_{\beta, \beta_0} \left\{ \frac{1}{\|\beta\|} \min_i [y_i(\beta^T \mathbf{x}_i + \beta_0)] \right\} \tag{1}$$

9.1 Maximal Margin Classifier

Computing the max margin hyperplane

Note that rescaling $\beta \to \kappa\beta$ and $\beta_0 \to \kappa\beta_0$ does not change (1).

Therefore, we can consider

$$\text{maximize}_{\beta_0,\beta_1,\ldots,\beta_p} M$$

$$\text{subject to } \sum_{j=1}^{p} \beta_j^2 = 1,$$

$$\text{and } y_i(\beta_0 + \beta_1 x_{i1} + \ldots + \beta_p x_{ip}) \geq M \text{ for all } i$$

9.1 Maximal Margin Classifier

Computing the max margin hyperplane

Remarks

Note that $M > 0$ is the half of the width of the strip separating the two classes.

The eventual solution, the max margin hyperplane is determined by the support vectors.
If x_i on the correct side of the trip varies, the solution would remain same.

The max margin hyperplane may vary a lot when the support vectors vary. (high variance; see Figure 9.5)

Primal-dual Support Vector Classifier

Equivalent reformulation of hard margin

$$\text{maximize}_{\beta_0,\beta_1,\ldots,\beta_p} M$$

$$\text{subject to } \sum_{j=1}^{p} \beta_j^2 = 1,$$

$$\text{and } y_i(\beta_0 + \beta_1 x_{i1} + \ldots + \beta_p x_{ip}) \geq M \text{ for all } i$$

$$\Leftrightarrow$$

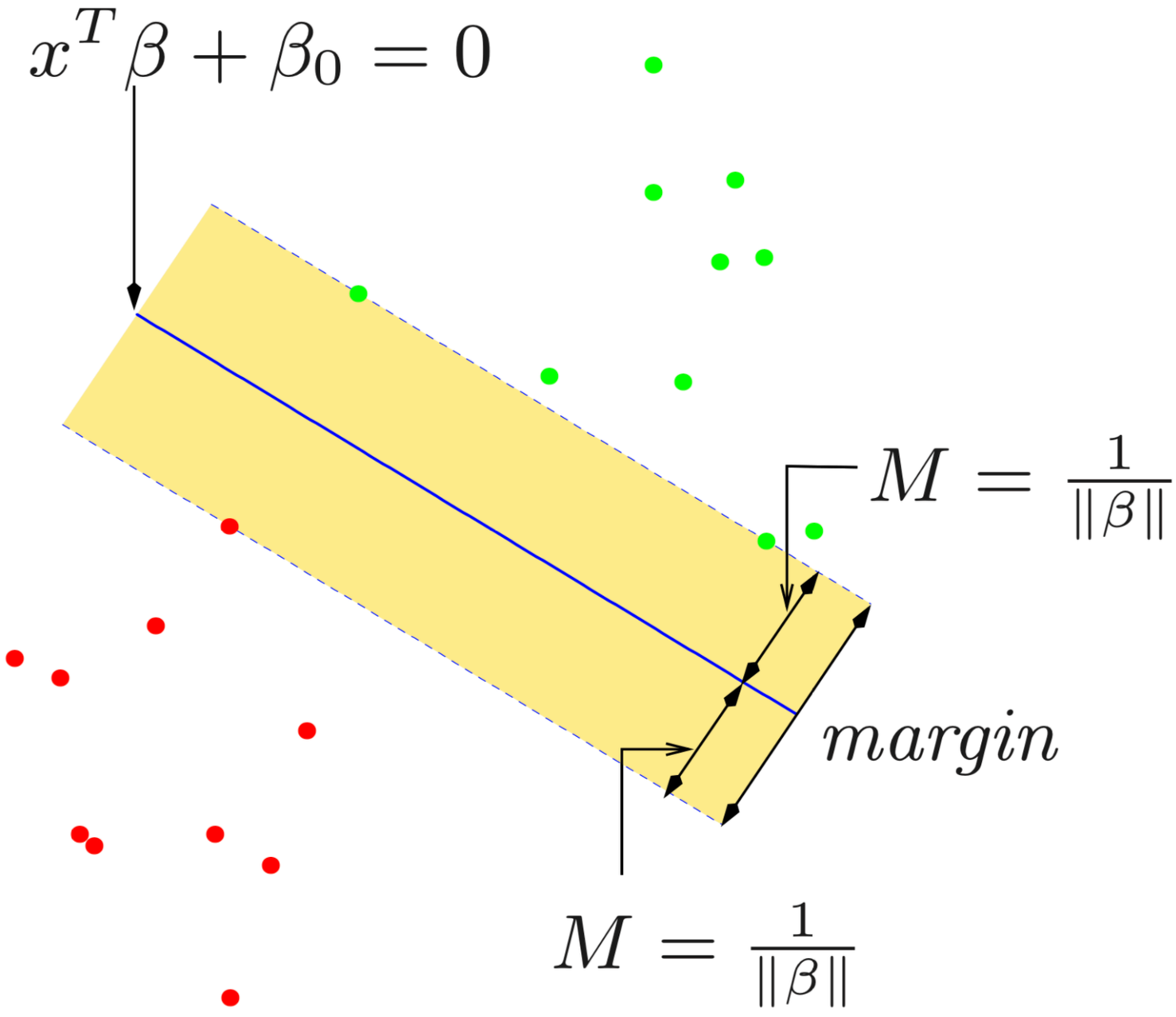
$$\text{minimize}_{\beta_0,\beta_1,\ldots,\beta_p} \|\beta\|^2 := \sum_j \beta_j^2$$

$$\text{subject to } y_i(\beta_0 + \beta_1 x_{i1} + \ldots + \beta_p x_{ip}) \geq 1 \text{ for all } i \text{ ,}$$

using $M = 1/\|\beta\|$.

Primal-dual Support Vector Classifier

Equivalent reformulation of hard margin



Primal-dual Support Vector Classifier

Lagrangian

Lagrangian: for $\alpha_i \geq 0$,

$$\max_{\alpha \geq 0} \min_{\beta} L(\beta, \alpha) := \frac{1}{2}\|\beta\|^2 - \sum_{i=1}^{n} \alpha_i (y_i(\beta_0 + \beta_1 x_{i1} + ... + \beta_p x_{ip}) - 1)$$

So

$$0 = \frac{\partial L}{\partial \beta} \Rightarrow \hat{\beta} = \sum_i \alpha_i y_i \mathbf{x}_i$$

$$0 = \frac{\partial L}{\partial \beta_0} \Rightarrow \sum_i \hat{\alpha}_i y_i = 0$$

and complementary condition

$$\hat{\alpha}_i (y_i(\hat{\beta}_0 + \left\langle \hat{\beta}, \mathbf{x}_i \right\rangle) - 1) = 0, \quad \text{for all } i$$

Primal-dual Support Vector Classifier

Support Vectors

Complementary condition $\hat{\alpha}_i(y_i(\hat{\beta}_0 + \left\langle \hat{\beta}, \mathbf{x}_i \right\rangle) - 1) = 0, \quad$ for all i implies that

$$y_i(\hat{\beta}_0 + \left\langle \hat{\beta}, \mathbf{x}_i \right\rangle) > 1 \Rightarrow \hat{\alpha}_i = 0$$

$$y_i(\hat{\beta}_0 + \left\langle \hat{\beta}, \mathbf{x}_i \right\rangle) = 1 \Rightarrow \hat{\alpha}_i \geq 0$$

Those sample point i's such that $\hat{\alpha}_i > 0$, are called *support vectors* (sv), which decided the maximal margin hyperplane

$$\hat{\beta} = \sum_{i \in sv} \hat{\alpha}_i y_i \mathbf{x}_i$$

and $\hat{\beta}_0$ can be uniquely decided by any support vector s using $\hat{\beta}_0 = y_s - \left\langle \hat{\beta}, \mathbf{x}_s \right\rangle$.

Primal-dual Support Vector Classifier

Dual formulation

After plugging $\hat{\beta}$ in the Lagrangian, it gives

$$\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{n} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$

subject to

$$\alpha_i \geq 0$$

$$\sum_i \alpha_i y_i = 0$$

9.1 Maximal Margin Classifier

Computing the max margin hyperplane

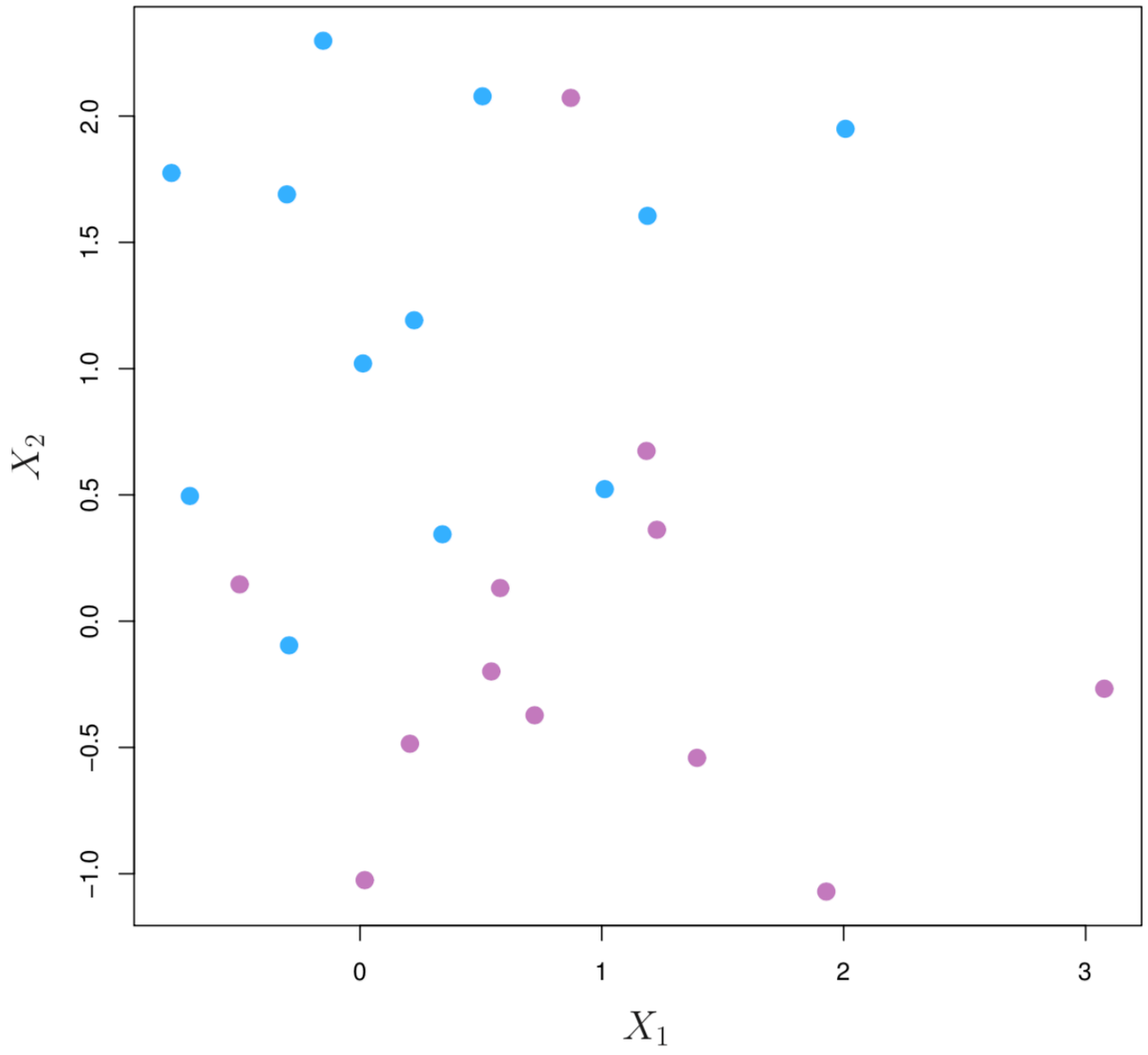


Figure: 9.4. There are two classes of observations, shown in blue and in purple. In this case, the two classes are not separable by a hyperplane, and so the maximal margin classifier cannot be used.

9.2 Support Vector Classifiers

Non-separable case

In general, the two classes are usually not separable by any hyperplane.

Even if they are, the max margin may not be desirable because of its high variance, and thus possible over-fit.

The generalization of the maximal margin classifier to the non-separable case is known as the support vector classifier.

Use a soft-margin in place of the max margin

9.2 Support Vector Classifiers

Non-separable case

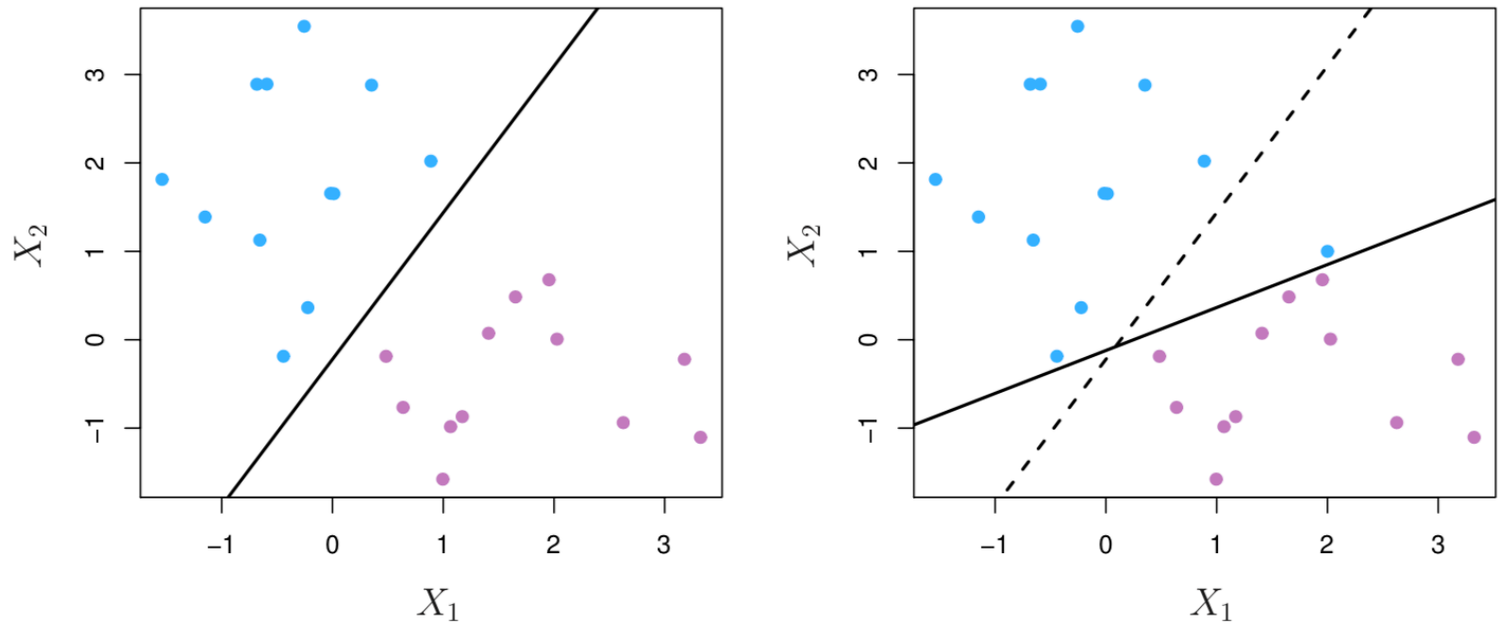


Figure: 9.5. Left: Two classes of observations are shown in blue and in purple, along with the maximal margin hyperplane. Right: An additional blue observation has been added, leading to a dramatic shift in the maximal margin hyperplane shown as a solid line. The dashed line indicates the maximal margin hyperplane that was obtained in the absence of this additional point.

9.2 Support Vector Classifiers

Non-perfect separation

Consider a classifier based on a hyperplane that does not perfectly separate the two classes, in the interest of

- *Greater robustness to individual observations, and*
- *Better classification of most of the training observations.*

Soft-margin classifier (support vector classifier) allow some violation of the margin: some can be on the wrong side of the margin (in the river) or even wrong side of the hyperplane; see Figure 9.6.

9.2 Support Vector Classifiers

Non-perfect separation

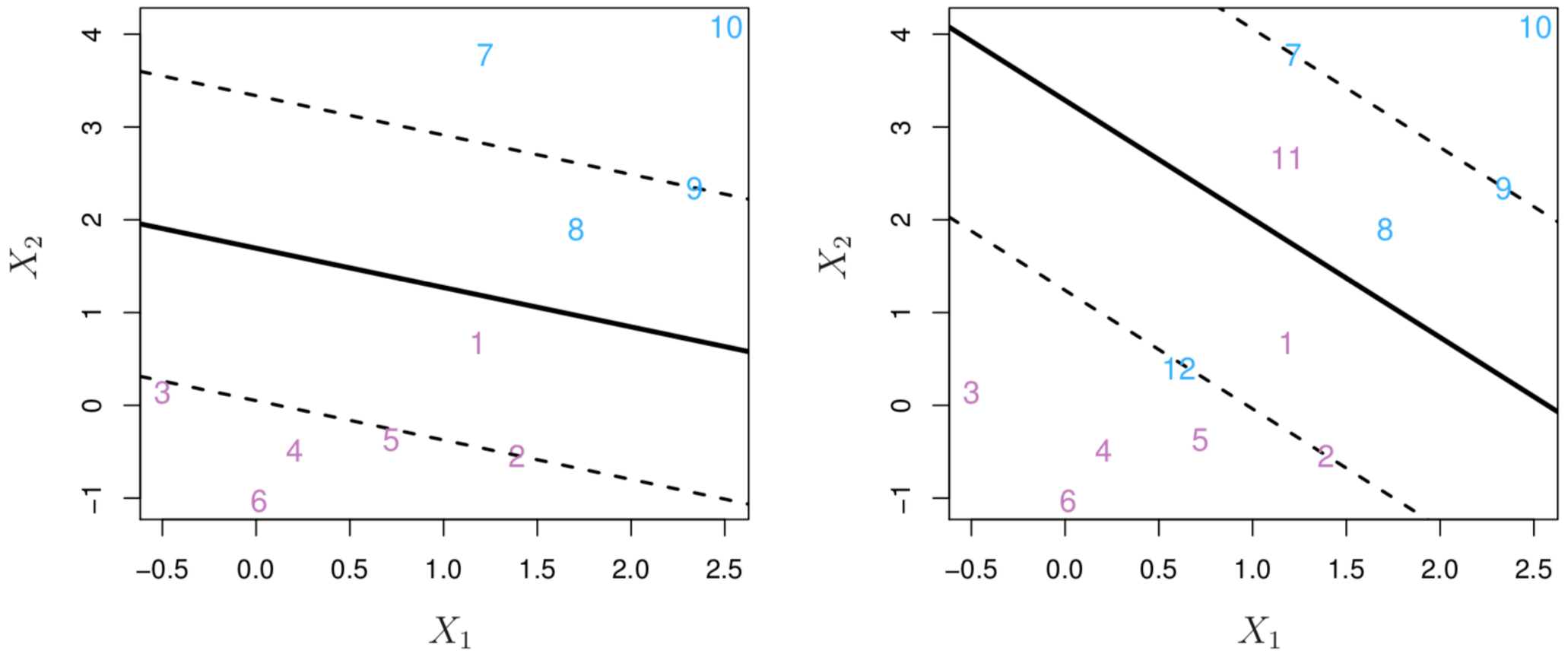


FIGURE 9.6. Left: A support vector classifier was fit to a small data set. The hyperplane is shown as a solid line and the margins are shown as dashed lines. Purple observations: Observations 3, 4, 5, and 6 are on the correct side of the margin, observation 2 is on the margin, and observation 1 is on the wrong side of the margin. Blue observations: Observations 7 and 10 are on the correct side of the margin, observation 9 is on the margin, and observation 8 is on the wrong side of the margin. No observations are on the wrong side of the hyperplane. Right: Same as left panel with two additional points, 11 and 12. These two observations are on the wrong side of the hyperplane and the wrong side of the margin.

9.2 Support Vector Classifiers

Computing the support vector classifier

Formulation

$$\text{maximize}_{\beta_0,\beta_1,\ldots,\beta_p} M$$

$$\text{subject to} \quad \sum_{j=1}^{p} \beta_j^2 = 1, \quad \text{and}$$

$$y_i(\beta_0 + \beta_1 x_{i1} + \ldots + \beta_p x_{ip}) \geq M(1 - \epsilon_i), \;\; \epsilon_i \geq 0 \text{ for all } i$$

$$\text{and} \sum_{i=1}^{n} \epsilon_i \leq C,$$

where C is a nonnegative tuning parameter. ϵ_i are *slack variables.*

9.2 Support Vector Classifiers

Computing the support vector classifier

Prediction

The solution of that optimization is the support vector classifier:

$$f(\mathbf{x}) = \beta_0 + \beta_1 x_1 + ... + \beta_p x_p.$$

And we classify an observation in $+1$ class if $f(\mathbf{x}) > 0$; else into -1 class.

9.2 Support Vector Classifiers

Understanding the slack variable ϵ_i

$\epsilon_i = 0 \iff$ the i-th observation is on the correct side of the margin

$\epsilon_i > 0 \iff$ the i-th observation is on the wrong side of the margin

$\epsilon_i > 1 \iff$ the i-th observation is on the wrong side of the hyperplane.

9.2 Support Vector Classifiers

Understanding tuning parameter C

C is a *budget* for the amount that the margin can be violated by the n observations

$C = 0 \iff$ no budget
As a result $\epsilon_i = 0$ for all i.
The classifier is a maximal margin classifier, which exists only if the two classes are separable by hyperplanes.

Larger C, more tolerance of margin violation.

No more than C observations can be on the wrong side of the soft-margin classifier hyperplane.

As C increases, the margin widens and more violations of the margin.

9.2 Support Vector Classifiers

Understanding tuning parameter C

More Remarks

C controls the bias-variance trade-off.

Small C high variance, small bias.

Large C: small variance, high bias.

C can be determined by using cross validation.

9.2 Support Vector Classifiers

Support vectors

Remarks

Observations that lie directly on the margin, or on the wrong side of the margin for their class, are known as support vectors.

Only the support vectors affect the support vector classifier.

Those strictly on the correct side of the margin do not. (robustness, analogous to median)

Larger C ⟹ more violations, ⟹ more support vectors, ⟹ smaller variance and more robust classifier.

9.2 Support Vector Classifiers

Support vectors

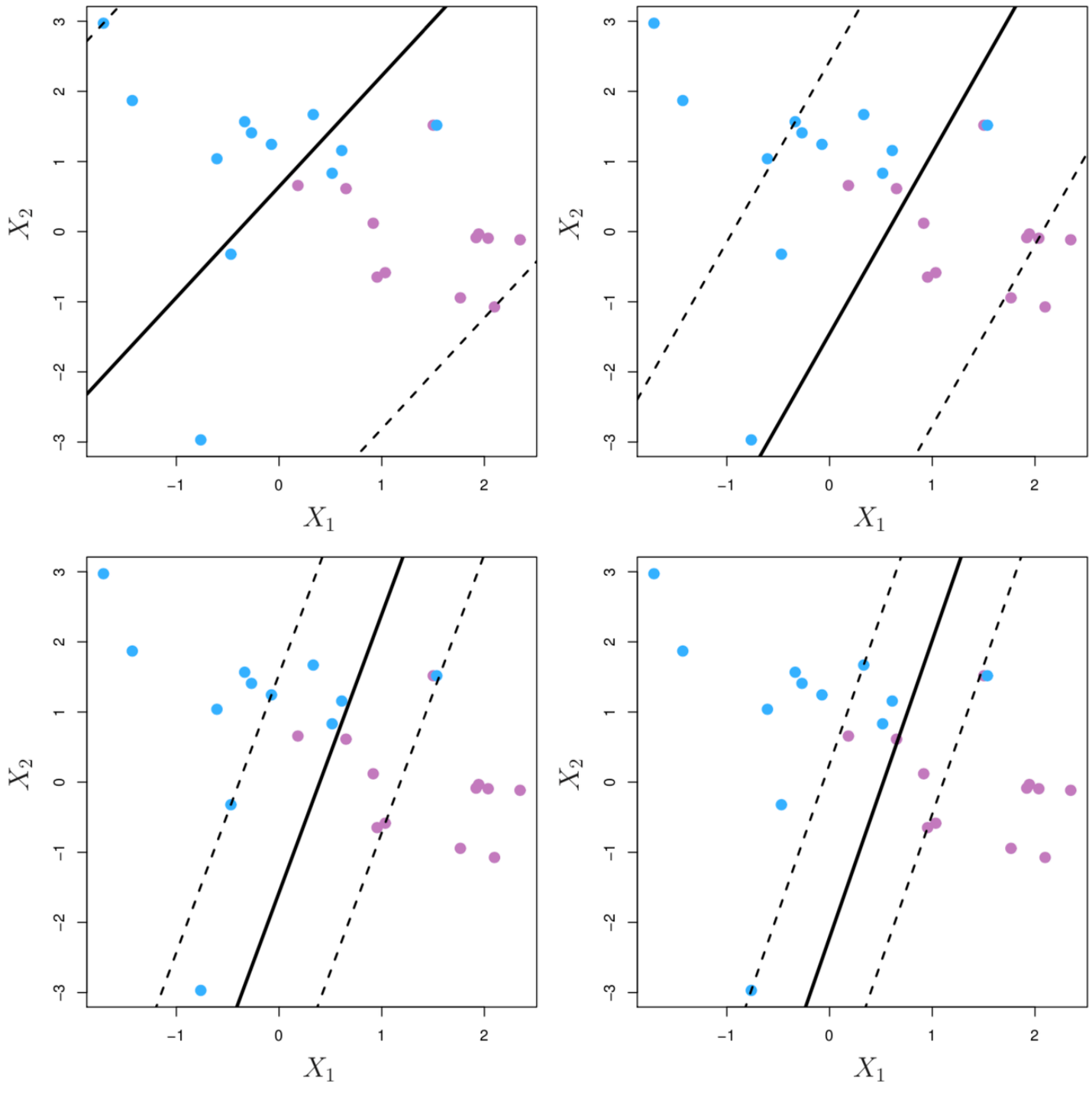


Figure: 9.7.

details on next slide

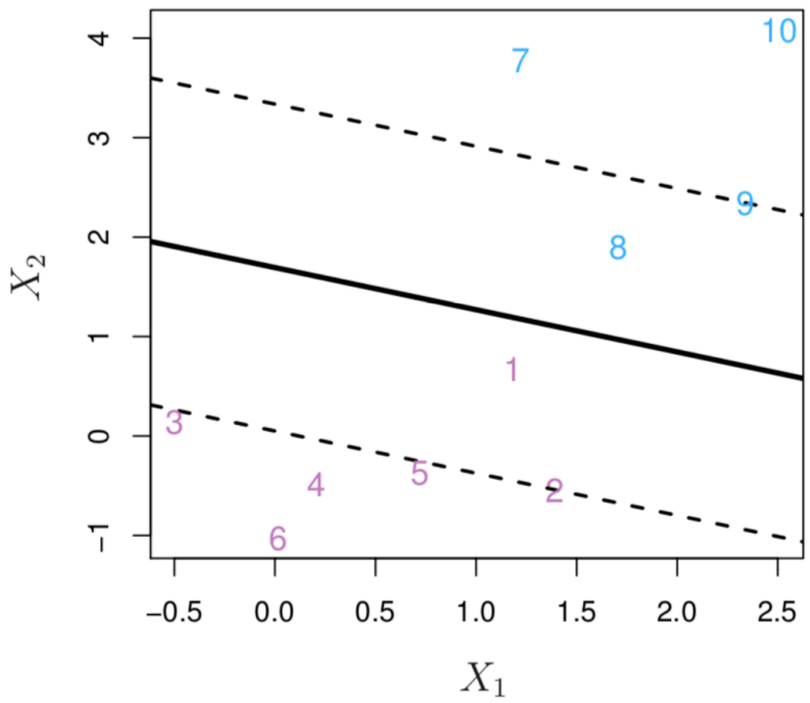
9.2 Support Vector Classifiers

Support vectors

Figure 9.7. A support vector classifier was fit using four different values of the tuning parameter C in (9.12) and (9.15). The largest value of C was used in the top left panel, and smaller values were used in the top right, bottom left, and bottom right panels. When C is large, then there is a high tolerance for observations being on the wrong side of the margin, and so the margin will be large. As C decreases, the tolerance for observations being on the wrong side of the margin decreases, and the margin narrows.

9.2 Support Vector Classifiers

Support vectors



Which observations are support vectors in the above example?

(a) 1,2,8,9

(b) 2,5,9

(c) 1,8

Primal-dual Support Vector Classifier

Equivalent reformulation of Soft-margin Classifier

$$\text{maximize}_{\beta_0,\beta_1,\ldots,\beta_p} M$$

$$\text{subject to } \sum_{j=1}^{p} \beta_j^2 = 1,$$

$$\text{and } y_i(\beta_0 + \beta_1 x_{i1} + \ldots + \beta_p x_{ip}) \geq M(1-\epsilon_i),\ \epsilon_i \geq 0,\ \text{for all } i$$

$$\text{and } \sum_{i=1}^{n} \epsilon \leq C,$$

$$\Leftrightarrow$$

$$\text{minimize}_{\beta_0,\beta_1,\ldots,\beta_p} \|\beta\|^2 + C\sum_{i=1}^{n} \xi_i$$

$$\text{subject to } y_i(\beta_0 + \beta_1 x_{i1} + \ldots + \beta_p x_{ip}) \geq 1 - \xi_i,\ \xi_i \geq 0 \text{ for all } i\ ,$$

Primal-dual Support Vector Classifier

Equivalent reformulation of Soft-margin Classifier

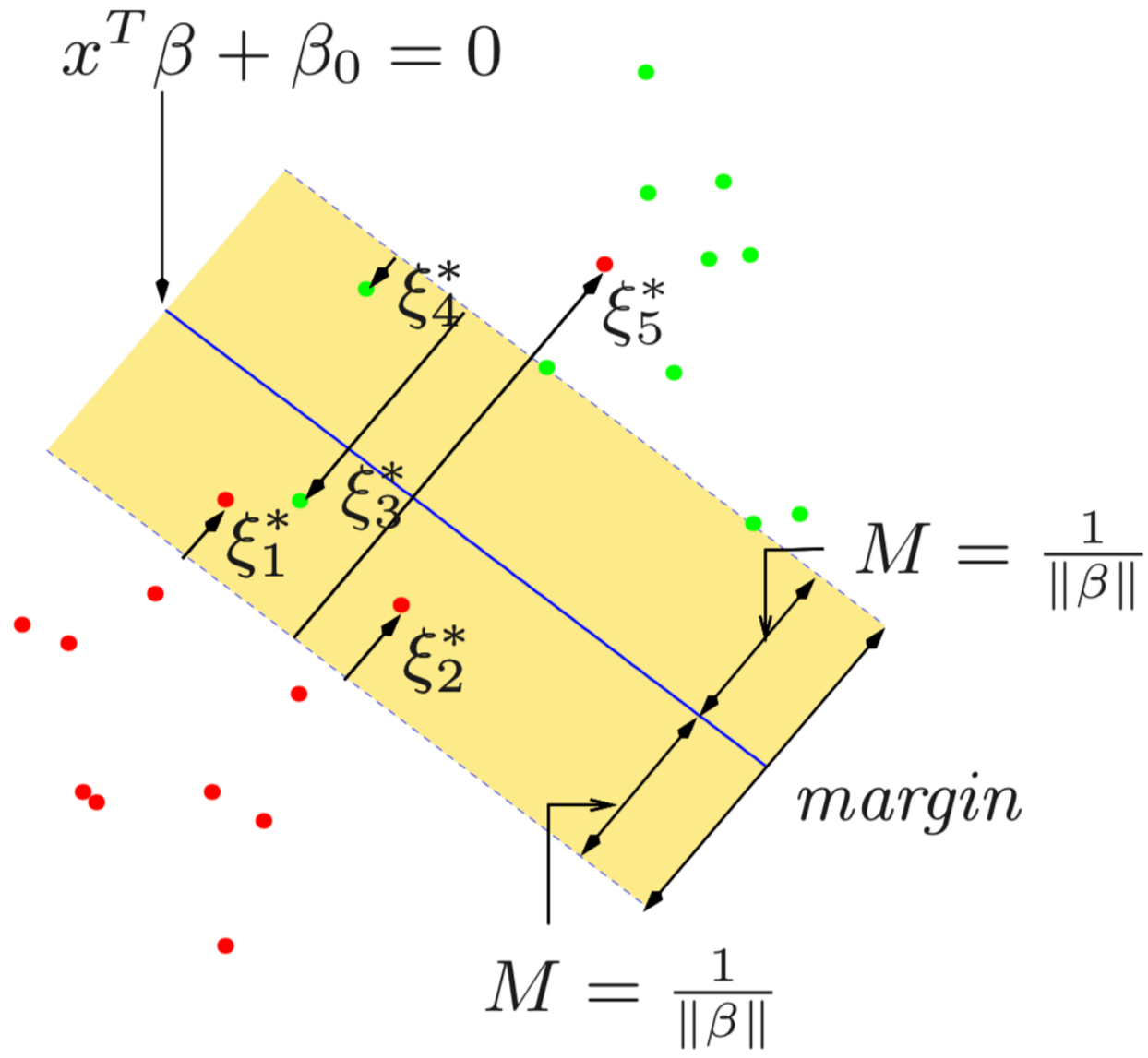


Figure: Separating hyperplane with margin

Primal-dual Support Vector Classifier

Equivalent reformulation of Soft-margin Classifier

Lagrangian for $\alpha_i \geq 0$, $\mu_i \geq 0$, $\xi_i \geq 0$,

$$L(\beta, \xi, \alpha, \mu) = \frac{1}{2}\|\beta\|^2 + C\sum_{i=1}^{n}\xi_i - \sum_{i=1}^{n}\alpha_i[y_i(\beta_0 + \mathbf{x}_i^T\beta) - (1-\xi_i)] - \sum_{i=1}^{n}\mu_i\xi_i,$$

So for all i,

$$0 = \frac{\partial L}{\partial \beta} \Rightarrow \hat{\beta} = \sum_i \hat{\alpha}_i y_i \mathbf{x}_i \tag{2}$$

$$0 = \frac{\partial L}{\partial \beta_0} \Rightarrow \sum_i \hat{\alpha}_i y_i = 0 \tag{3}$$

$$0 = \frac{\partial L}{\partial \xi_i} \Rightarrow \hat{\alpha}_i + \mu_i = C \tag{4}$$

and complementary condition

$$\hat{\alpha}_i[y_i(\hat{\beta}_0 + \mathbf{x}_i^T\hat{\beta}) - (1-\xi_i)] = 0$$

$$\mu_i\xi_i = 0$$

Primal-dual Support Vector Classifier

Equivalent reformulation of Soft-margin Classifier

Support vectors

Complementary condition $\hat{\alpha}_i[y_i(\hat{\beta}_0 + \mathbf{x}_i^T\hat{\beta}) - (1-\xi_i)] = 0$, $\xi_i \geq 0$, $\Rightarrow$

$$y_i(\hat{\beta}_0 + \left\langle \hat{\beta}, \mathbf{x}_i \right\rangle) > 1 \Rightarrow \hat{\alpha}_i = 0$$

$$y_i(\hat{\beta}_0 + \left\langle \hat{\beta}, \mathbf{x}_i \right\rangle) = 1 - \xi_i \Rightarrow C \geq \hat{\alpha}_i \geq 0$$

$$\mu_i\xi_i = 0 \Rightarrow \xi_i > 0, \mu_i = 0, \hat{\alpha}_i = C - \mu_i = C$$

Those samples such that $C \geq \hat{\alpha}_i > 0$, are called *support vectors* (sv),

$$\hat{\beta} = \sum_{i \in sv} \hat{\alpha}_i y_i \mathbf{x}_i$$

and those s.t. $\alpha_i = C$ are within margin (violators), $\xi_i = 0$ are on margin (deciding $\hat{\beta}_0$).

Primal-dual Support Vector Classifier

Dual form of Soft-margin Classifier

Substituting (2)-(4) into the Lagrangian, it gives the dual optimization problem

$$\min_{\alpha} \frac{1}{2} \sum_{i,j=1}^{n} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j - \sum_{i=1}^{n} \alpha_i$$

subject to

$$C \geq \alpha_i \geq 0$$

$$\sum_i \alpha_i y_i = 0$$

9.3 Support Vector Machines

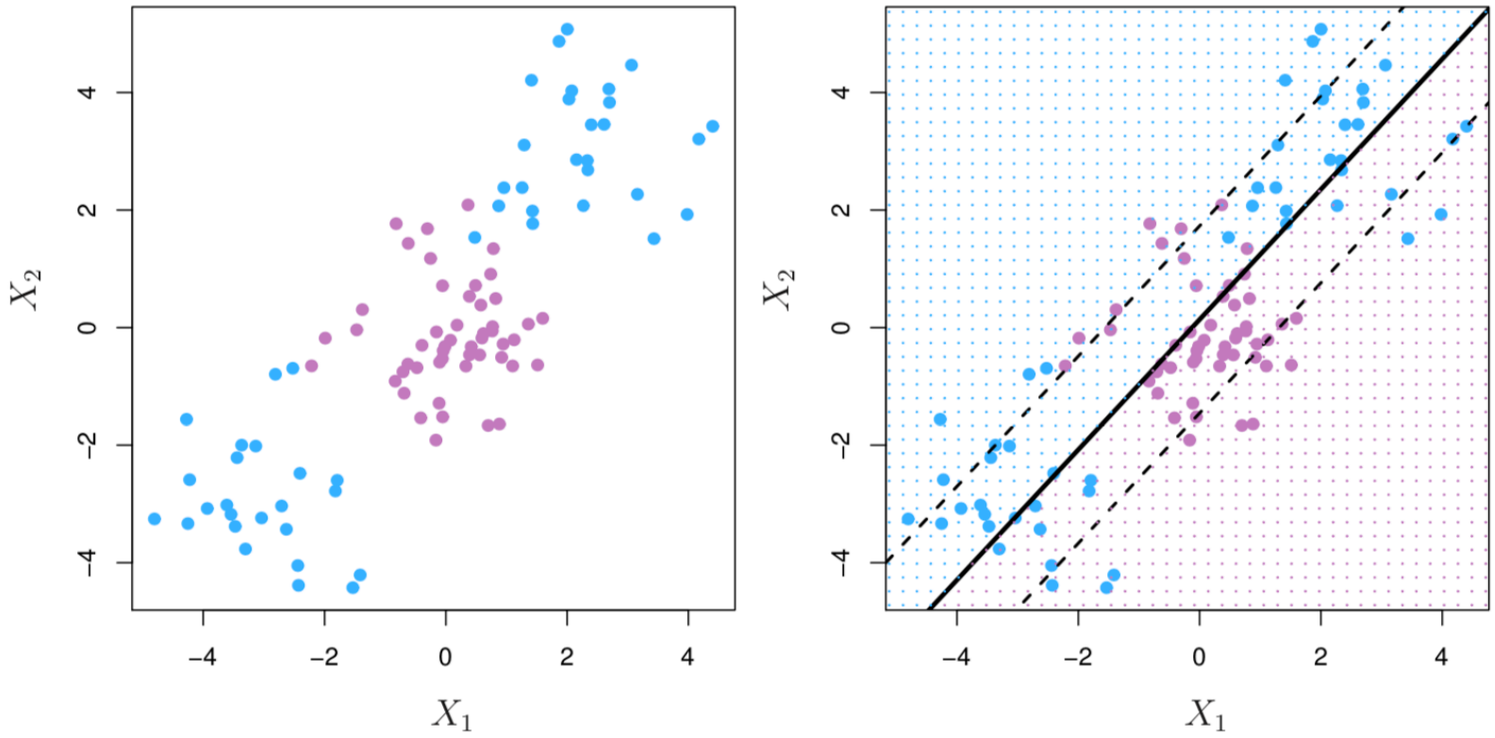


Figure: 9.8. Nonlinear boundaries. Left: The observations fall into two classes, with a non-linear boundary between them. Right: The support vector classifier seeks a linear boundary, and consequently performs very poorly.

9.3 Support Vector Machines

Extending to nonlinear boundary

In practice, we are sometimes faced with non-linear class boundaries

Linear classifier could perform poorly.

Need nonlinear classifier.

As in the extension to the polynomial regression from linear regression, we can consider enlarge the feature space from the original p inputs to polynomials (of certain order) of the inputs.

9.3 Support Vector Machines

Extending to quadratic inputs

Rather than constructing the support vector classifier using p features:

$$X_1, ..., X_p.$$

we use $2p$ features:

$$X_1, X_1^2, ..., X_p, X_p^2.$$

Treat them as $2p$ original inputs, and fit the support vector classifier.

The separating hyperplane is a hyperplane in R^{2p}, which should be a linear equation:

$$\beta_0 + \beta_1 X_1 + \beta_2 X_p + \beta_{p+1} X_1^2 + ... + \beta_{2p} X_p^2 = 0$$

This is a quadrtic equation in $X_1, ..., X_p$. Thus the separating surface in R^p in terms of $X_1, ..., X_p$ corresponds to a quadratic surface in R^p.

9.3 Support Vector Machines

Extending to polynomial inputs

Remarks

Can extend to polynomial of any given order d.

Could lead to too many features, too large feature space; thus overfit.

Higher powers are unstable.

9.3 Support Vector Machines

Key observation

The linear support vector classifier can be represented as

$$f(\mathbf{x}) = \beta_0 + \sum_{i=1}^{n} a_i \langle \mathbf{x}_i, \mathbf{x} \rangle$$

$a_i \neq 0$ only for all support vectors.

a_i can also be computed based on $\langle \mathbf{x}_j, \mathbf{x}_k \rangle$ and y_i.

Only the inner product of the feature space is relevant in computing the linear support vector classifier.

9.3 Support Vector Machines

Kernel trick

The inner product $\langle \cdot, \cdot \rangle$ is a bivariate function (satisfying some property).

It can be genearlized to kernel functions

$$K(\mathbf{x}, \mathbf{z})$$

which is positive definite.

The classifier can be expressed as

$$f(\mathbf{x}) = \beta_0 + \sum_{i=1}^{n} a_i K(\mathbf{x}, \mathbf{x}_i)$$

9.3 Support Vector Machines

Examples of kernels

linear kernel

$$K(x_i, x_j) = \langle x_i, x_j \rangle = x_i^T x_j.$$

polynomial kernel of degree d:

$$K(x_i, x_j) = (1 + \langle x_i, x_j \rangle)^d.$$

Gaussian radial kernel:

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2), \quad \gamma > 0.$$

Only the inner product of the feature space is relevant in computing the linaer support vector classfier.

9.3 Support Vector Machines

Examples of kernels

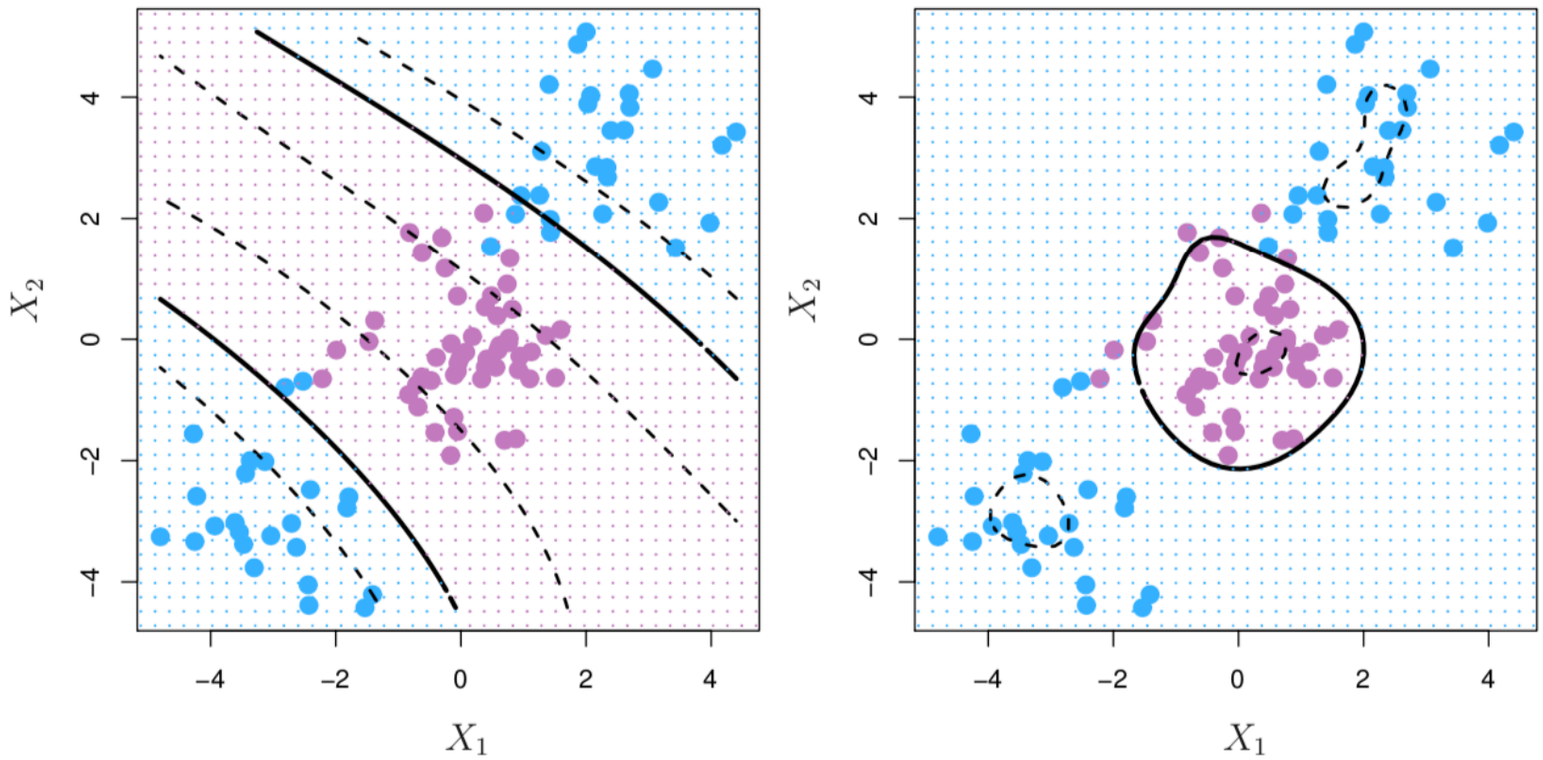


Figure: 9.9. Left: An SVM with a polynomial kernel of degree 3 is applied to the non-linear data from Figure 9.8, resulting in a far more appropriate decision rule. Right: An SVM with a radial kernel is applied. In this example, either kernel is capable of capturing the decision boundary.

9.3 Support Vector Machines

Radial kernel

The radial kernel has local behavior.

To predict the class for a new observation with input $\mathbf{x}$,

$$f(\mathbf{x}) = \beta_0 + \sum_{i=1}^{n} a_i \exp(-\gamma \|\mathbf{x} - \mathbf{x}_i\|^2)$$

A training data point $\mathbf{x}_i$ being far away from $\mathbf{x}$ will have little effect to the sign of $f(\mathbf{x})$.

9.3 Support Vector Machines

The enlarged feature space

The support vector machine actually enlarges the original feature space to a space of kernel functions.

$$\mathbf{x_i} \to K(\cdot, \mathbf{x}_i).$$

The original space of p inputs has dimension p.

The enlarged space of features, the function space, is infinite dimension!

In actual fitting of the support vector machine, we only need to compute the $K(x_i, x_j)$ for all x_i, x_j in training data.

Do not have to work with the enlarged feature space of infinite dimension.

9.3 Support Vector Machines

Example: the Heart data

```
> ######################################################  Heart data
> Heart<-read.csv("Heart.csv",header=TRUE)
> Heart[1:10,]
   Age Sex    ChestPain RestBP Chol Fbs RestECG MaxHR ExAng Oldpeak Slope Ca       Thal AHD
1   63   1      typical    145  233   1       2   150     0     2.3     3  0      fixed  No
2   67   1 asymptomatic    160  286   0       2   108     1     1.5     2  3     normal Yes
3   67   1 asymptomatic    120  229   0       2   129     1     2.6     2  2 reversable Yes
4   37   1   nonanginal    130  250   0       0   187     0     3.5     3  0     normal  No
5   41   0   nontypical    130  204   0       2   172     0     1.4     1  0     normal  No
6   56   1   nontypical    120  236   0       0   178     0     0.8     1  0     normal  No
7   62   0 asymptomatic    140  268   0       2   160     0     3.6     3  2     normal Yes
8   57   0 asymptomatic    120  354   0       0   163     1     0.6     1  0     normal  No
9   63   1 asymptomatic    130  254   0       2   147     0     1.4     2  1 reversable Yes
10  53   1 asymptomatic    140  203   1       2   155     1     3.1     3  0 reversable Yes
```

9.3 Support Vector Machines

Example: the Heart data

In Chapter 8 we apply decision trees and related methods to the Heart data.

The aim is to use 13 predictors such as Age, Sex, and Chol in order to predict whether an individual has heart disease.

We now investigate how an SVM compares to LDA on this data.

297 subjects, randomly split into 207 training and 90 test observations.

9.3 Support Vector Machines

Support Vector Classifier in R

Example: the Heart data

```
> Heart<-read.csv("Heart.csv",header=TRUE)
> Heart<-na.omit(Heart)
> Heart[1:10,]
   Age Sex    ChestPain RestBP Chol Fbs RestECG MaxHR ExAng Oldpeak Slope Ca       Thal AHD
1   63   1      typical    145  233   1       2   150     0     2.3     3  0      fixed  No
2   67   1 asymptomatic    160  286   0       2   108     1     1.5     2  3     normal Yes
3   67   1 asymptomatic    120  229   0       2   129     1     2.6     2  2 reversable Yes
4   37   1   nonanginal    130  250   0       0   187     0     3.5     3  0     normal  No
5   41   0   nontypical    130  204   0       2   172     0     1.4     1  0     normal  No
6   56   1   nontypical    120  236   0       0   178     0     0.8     1  0     normal  No
7   62   0 asymptomatic    140  268   0       2   160     0     3.6     3  2     normal Yes
8   57   0 asymptomatic    120  354   0       0   163     1     0.6     1  0     normal  No
9   63   1 asymptomatic    130  254   0       2   147     0     1.4     2  1 reversable Yes
10  53   1 asymptomatic    140  203   1       2   155     1     3.1     3  0 reversable Yes
> Heart$AHD<-as.factor(Heart$AHD)
> train<-sample(297,207)
>
> library(e1071)
> svmfit=svm(AHD~., data=Heart[train,], kernel="linear", cost=10,scale=FALSE)         ## Support Vector Classifier
> summary(svmfit)

Call:
svm(formula = AHD ~ ., data = Heart[train, ], kernel = "linear", cost = 10, scale = FALSE)


Parameters:
   SVM-Type:  C-classification
 SVM-Kernel:  linear
       cost:  10

Number of Support Vectors:  80

 ( 41 39 )


Number of Classes:  2

Levels:
 No Yes



> svmfit$index
 [1]   4  15  22  27  28  29  30  36  39  43  46  53  55  68  72  73  75  79  82  85 111 118 123 127 131 137 139 141 142 147 155 167 169 172 175 181 183 190 194 200 205
[42]   6  10  12  13  14  16  17  31  37  54  59  62  64  66  76  81  88  91  94  97 102 110 113 115 121 128 135 146 150 160 161 176 182 184 188 196 197 199 207
```

9.3 Support Vector Machines

Support Vector Classifier in R

Example: the Heart data

```
> ### prediction on test data
> ypred=predict(svmfit ,Heart[-train,])
> table(predict=ypred, truth=Heart[-train,]$AHD)
       truth
predict No Yes
    No  44   9
    Yes  1  36
> sum(ypred==Heart[-train,]$AHD)/90
[1] 0.8888889
```

```
> ######################## Use a different cost value
> svmfit=svm(AHD~., data=Heart[train,], kernel="linear", cost=5,scale=TRUE)          ## Support Vector Classifier
> ### prediction on test data
> ypred=predict(svmfit ,Heart[-train,])
> table(predict=ypred, truth=Heart[-train,]$AHD)
       truth
predict No Yes
    No  43   9
    Yes  2  36
> sum(ypred==Heart[-train,]$AHD)/90
[1] 0.8777778
```

9.3 Support Vector Machines

Support Vector Machine in R

Example: the Heart data

```
> ###################################  Support Vector Machine on Heart Data
> svmfit=svm(AHD~., data=Heart[train,], kernel="radial", gamma=1,cost =1)            ###### radial kernel with gamma = 1
> summary(svmfit)

Call:
svm(formula = AHD ~ ., data = Heart[train, ], kernel = "radial", gamma = 1, cost = 1)


Parameters:
   SVM-Type:  C-classification
 SVM-Kernel:  radial
       cost:  1

Number of Support Vectors:  207

 ( 115 92 )


Number of Classes:  2

Levels:
 No Yes



> svmfit$index
  [1]   1   2   3   4   7   8   9  15  19  21  22  26  27  28  29  30  32  35  36  39  40  43  44  45  46  49  50  51  53  55  56  57  58  63  68  70  71  72  73  75  77
 [42]  79  80  82  85  86  87  89  90  92  93  95  96 100 101 103 104 105 106 107 108 111 112 114 116 117 118 119 123 125 126 127 131 132 133 134 136 137 139 141 142 144
 [83] 145 147 148 149 152 153 154 155 156 158 162 164 165 167 169 171 172 173 175 180 181 183 185 186 187 189 190 192 194 198 200 203 205   5   6  10  11  12  13  14  16
[124]  17  18  20  23  24  25  31  33  34  37  38  41  42  47  48  52  54  59  60  61  62  64  65  66  67  69  74  76  78  81  83  84  88  91  94  97  98  99 102 109 110
[165] 113 115 120 121 122 124 128 129 130 135 138 140 143 146 150 151 157 159 160 161 163 166 168 170 174 176 177 178 179 182 184 188 191 193 195 196 197 199 201 202 204
[206] 206 207
```

9.3 Support Vector Machines

Support Vector Machine in R

Example: the Heart data

```
> ### prediction on test data
> ypred=predict(svmfit ,Heart[-train,])
> table(predict=ypred, truth=Heart[-train,]$AHD)
       truth
predict No Yes
    No  44  42
    Yes  1   3
> sum(ypred==Heart[-train,]$AHD)/90
[1] 0.5222222
```

```
> ###################################  Try different gamma values
> svmfit=svm(AHD~., data=Heart[train,], kernel="radial", gamma=0.1,cost =1)          ###### radial kernel with gamma = 1
> ### prediction on test data
> ypred=predict(svmfit ,Heart[-train,])
> table(predict=ypred, truth=Heart[-train,]$AHD)
       truth
predict No Yes
    No  42  12
    Yes  3  33
> sum(ypred==Heart[-train,]$AHD)/90
[1] 0.8333333
```

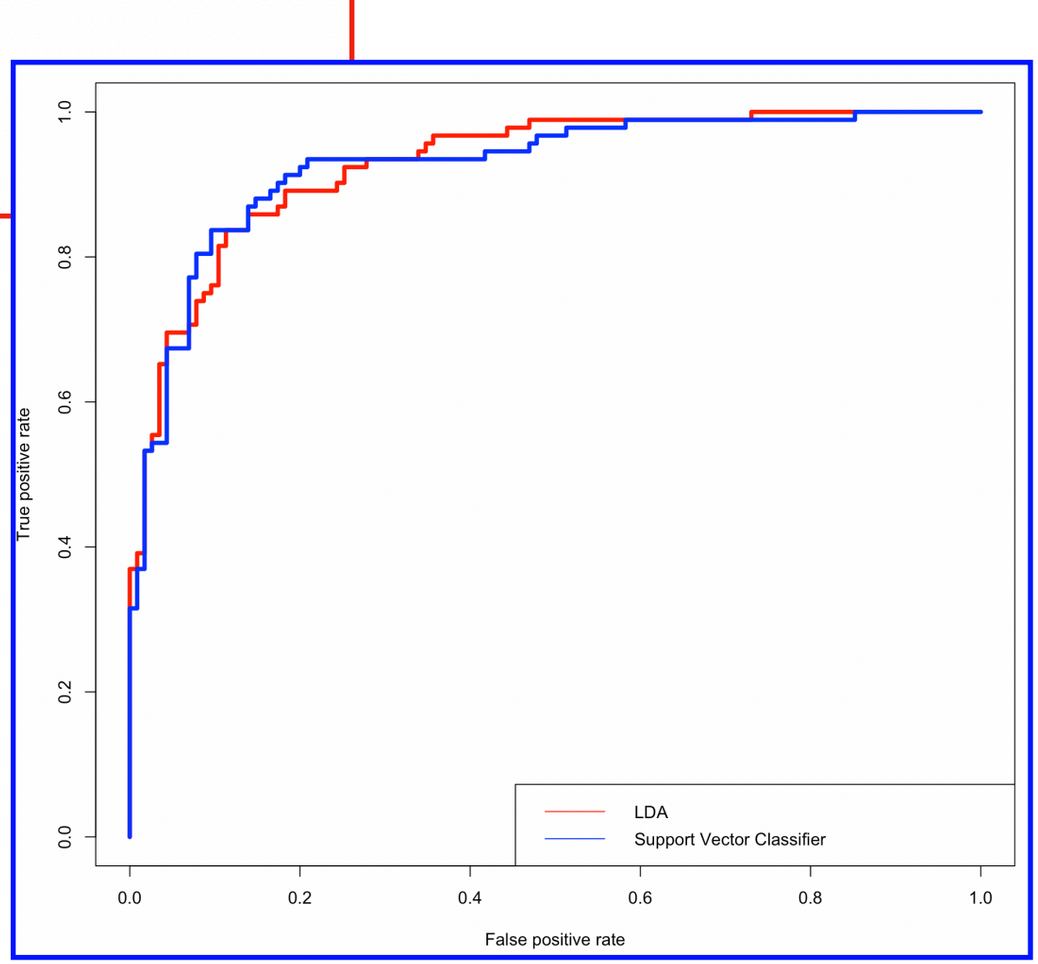
```
> ###################################  Try different gamma values
> svmfit=svm(AHD~., data=Heart[train,], kernel="radial", gamma=0.01,cost =2)          ###### radial kernel with gamma = 1
> ### prediction on test data
> ypred=predict(svmfit ,Heart[-train,])
> table(predict=ypred, truth=Heart[-train,]$AHD)
       truth
predict No Yes
    No  43   8
    Yes  2  37
> sum(ypred==Heart[-train,]$AHD)/90
[1] 0.8888889
```

9.3 Support Vector Machines

Example: the Heart data

ROC curves

```
###################################  ROC curves on training dataset:  LDA and Support Vector Classifier
library(MASS)                                                       ## load library
lda.fit<-lda(AHD~.,data=Heart[train,])
svmfit=svm(AHD~., data=Heart[train,], kernel="linear", cost=5,scale=TRUE)          ## Support Vector Classifier
library(ROCR)
lda.train<-predict(lda.fit,Heart[train,])
svmfit.train<-predict(svmfit,Heart[train,],decision.values=T)
lda.fit.pred<-prediction(lda.train$posterior[,2],Heart[train,]$AHD)
svmfit.pred<-prediction(-attributes(svmfit.train)$decision.values, Heart[train,]$AHD)
perf<-performance(lda.fit.pred,"tpr","fpr")
perfsvm<-performance(svmfit.pred,"tpr","fpr")
plot(perf,col="red",lwd=4)
plot(perfsvm,col="blue",lwd=4,add=TRUE)

legend("bottomright",
       legend = c("LDA", "Support Vector Classifier"),
       col = c("red","blue"),
       lty=c(1,1)
)
```

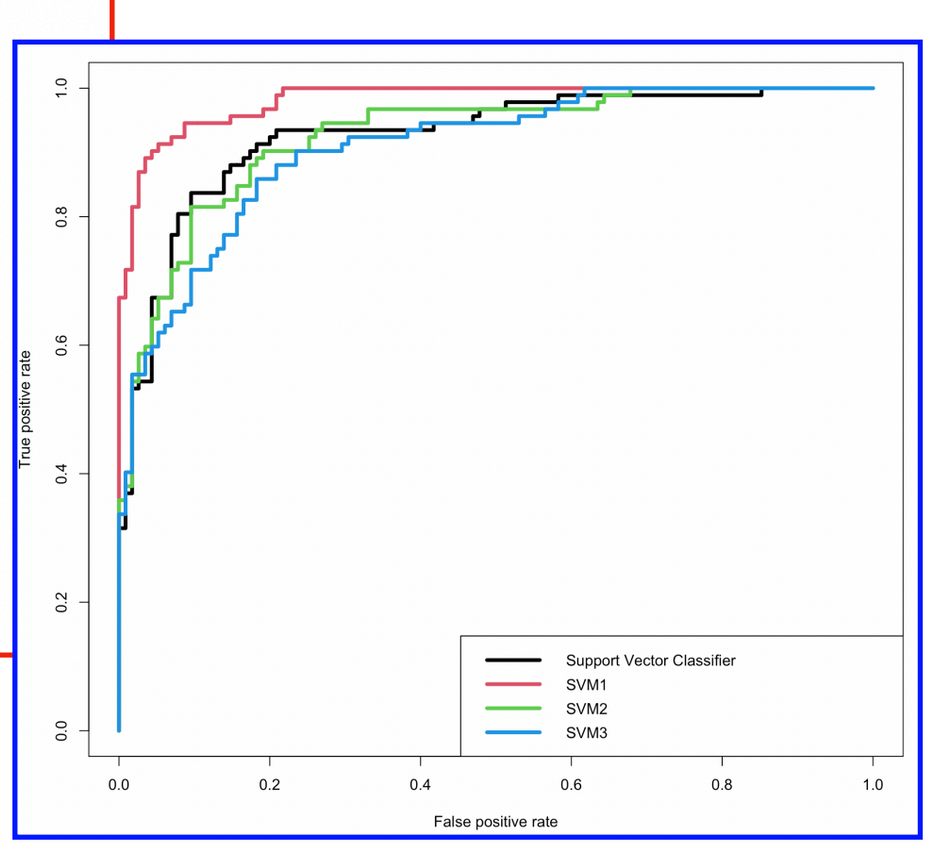


9.3 Support Vector Machines

ROC curves

Example: the Heart data

```
#####################################  ROC curves on training dataset:  LDA and Support Vector Classifier
library(ROCR)
svmfit.1=svm(AHD~., data=Heart[train,], kernel="linear", cost =5)
svmfit.train.1<-predict(svmfit.1,Heart[train,],decision.values=T)
svmfit.pred.1<-prediction(-attributes(svmfit.train.1)$decision.values, Heart[train,]$AHD)
perfsvm.1<-performance(svmfit.pred.1,"tpr","fpr")

svmfit.2=svm(AHD~., data=Heart[train,], kernel="radial",gamma=0.1,cost =1)
svmfit.train.2<-predict(svmfit.2,Heart[train,],decision.values=T)
svmfit.pred.2<-prediction(-attributes(svmfit.train.2)$decision.values, Heart[train,]$AHD)
perfsvm.2<-performance(svmfit.pred.2,"tpr","fpr")

svmfit.3=svm(AHD~., data=Heart[train,], kernel="radial",gamma=0.01,cost =1)
svmfit.train.3<-predict(svmfit.3,Heart[train,],decision.values=T)
svmfit.pred.3<-prediction(-attributes(svmfit.train.3)$decision.values, Heart[train,]$AHD)
perfsvm.3<-performance(svmfit.pred.3,"tpr","fpr")

svmfit.4=svm(AHD~., data=Heart[train,], kernel="radial",gamma=0.001,cost =1)
svmfit.train.4<-predict(svmfit.4,Heart[train,],decision.values=T)
svmfit.pred.4<-prediction(-attributes(svmfit.train.4)$decision.values, Heart[train,]$AHD)
perfsvm.4<-performance(svmfit.pred.4,"tpr","fpr")

plot(perfsvm.1,col=1,lwd=4)
plot(perfsvm.2,col=2,lwd=4,add=TRUE)
plot(perfsvm.3,col=3,lwd=4,add=TRUE)
plot(perfsvm.4,col=4,lwd=4,add=TRUE)

legend("bottomright",
       legend = c("Support Vector Classifier","SVM1","SVM2","SVM3"),
       col = c(1,2,3,4),
       lty=c(1,1,1,1),
       lwd=c(4,4,4,4)
)
```



9.3 Support Vector Machines

Example: the Heart data

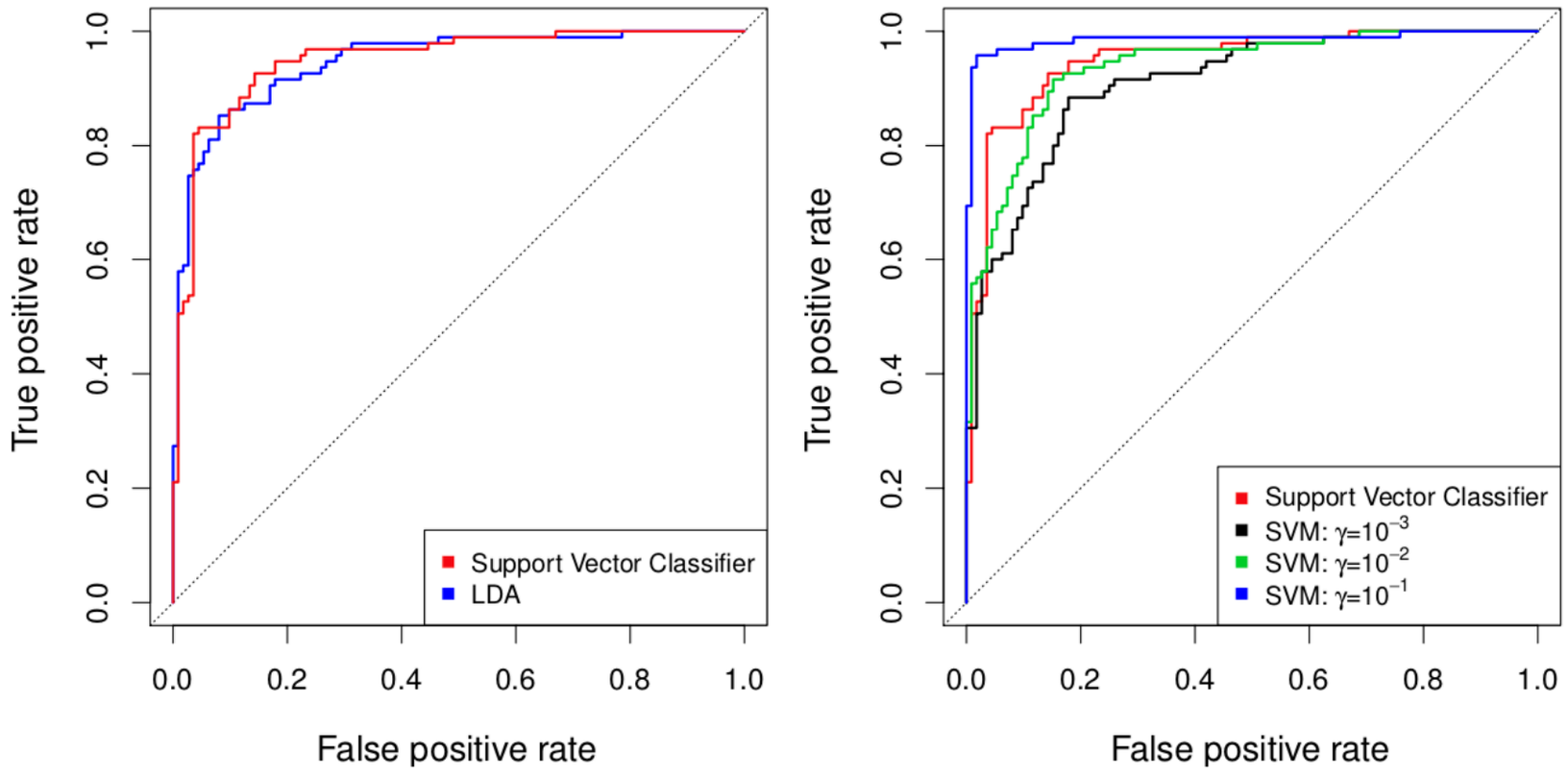


Figure: ROC curves for the Heart data training set. Left: The support vector classifier and LDA are compared. Right: The support vector classifier is compared to an SVM using a radial basis kernel with $\gamma = 10^{-3}, 10^{-2}$ and 10^{-1}.

9.3 Support Vector Machines

Example: the Heart data

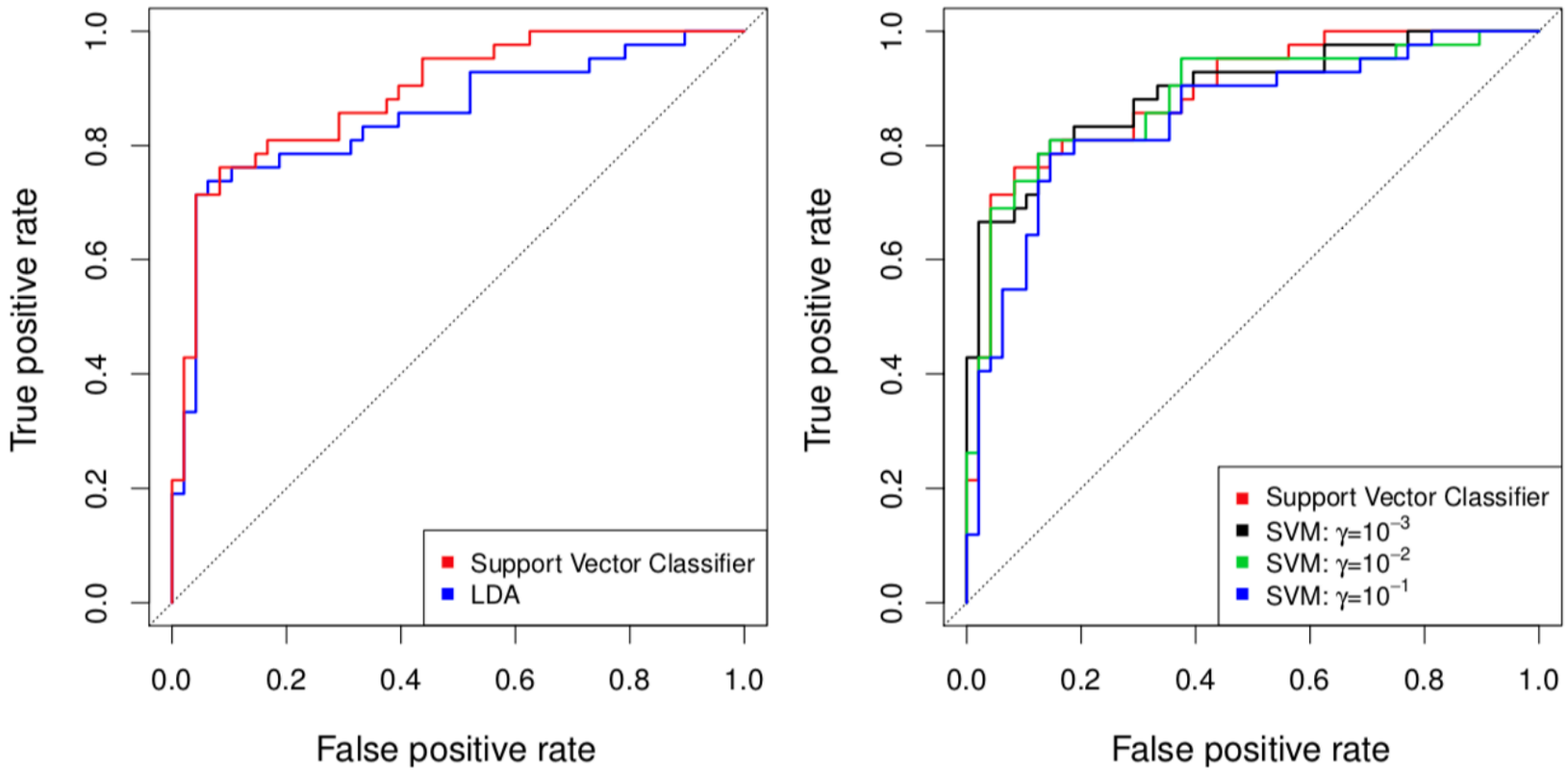


Figure: 9.11. ROC curves for the test set of the Heart data. Left: The support vector classifier and LDA are compared. Right: The support vector classifier is compared to an SVM using a radial basis kernel with $\gamma = 10^{-3}, 10^{-2}$ and 10^{-1}.

9.4 Primal-dual Support Vector Machine

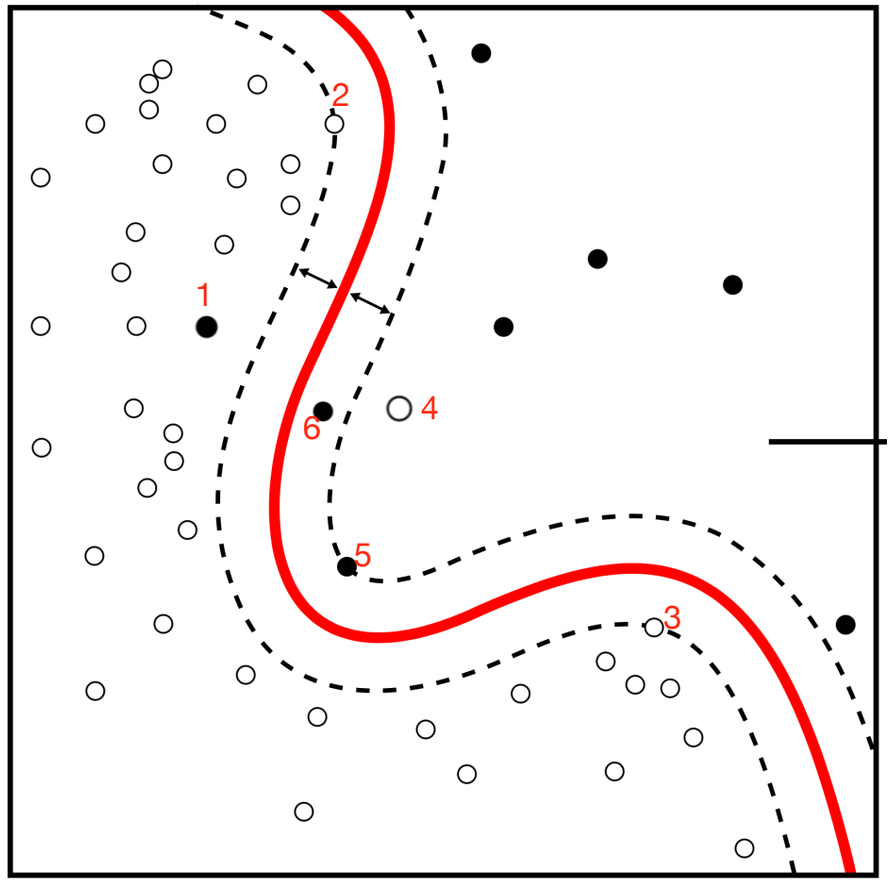
$$\min_{\alpha} \ \frac{1}{2}\sum_{i,j=1}^{n}\alpha_i\alpha_j y_i y_j K(\mathbf{x}_i,\mathbf{x}_j) - \sum_{i=1}^{n}\alpha_i$$

$$\text{subject to:} \quad C \geq \alpha_i \geq 0$$

$$\sum_{i=1}^{n}\alpha_i y_i = 0$$

9.4 Primal-dual Support Vector Classifier

Dual form of Soft-margin Classifier



Which observations are support vectors in the above example?

(a) 2,3,5

(b) 1,2,3,4,5,6

(c) 1,2,3,4,5

9.5. SVMs with More Than 2 Cases

One-versus-one approach

With $K > 2$ classes.

Run a SVM on each of the $\binom{K}{2}$ pairs of classes.

We obtain $\binom{K}{2}$ SVMs.

For every test observations, compute the number of times it is classified into class k by all the SVMs, denote as d_k.

Classify it into the class with highest d_k (majortiy vote).

9.5. SVMs with More Than 2 Cases

One-versus-all approach

With $K > 2$ classes.

Run a SVM on class k (coded as $+1$) versus class "not-k" (coded as -1): $f_k(\mathbf{x})$. (Note that the larger $f_k(\mathbf{x})$, the more likely $\mathbf{x}$ is in class k.

For a new test observation with $\mathbf{x}$, compute assign to the class with largest $f_k(\mathbf{x})$.

9.5. SVMs with More Than 2 Cases

Gene Expression Classification in R

We now examine the Khan data set, which consists of a number of tissue samples corresponding to four distinct types of small round blue cell tu- mors. For each tissue sample, gene expression measurements are available. The data set consists of training data, xtrain and ytrain, and testing data, xtest and ytest.

```
> library(ISLR)
> names(Khan)
[1] "xtrain" "xtest"  "ytrain" "ytest"
> dim(Khan$xtrain)
[1]   63 2308
> dim(Khan$xtest)
[1]   20 2308
> length(Khan$ytrain)
[1] 63
> length(Khan$ytest)
[1] 20
```

```
> table(Khan$ytrain)

 1  2  3  4
 8 23 12 20
> table(Khan$ytest)

1 2 3 4
3 6 6 5
```

```
> dat=data.frame(x=Khan$xtrain , y=as.factor(Khan$ytrain ))
> out=svm(y~., data=dat, kernel="linear",cost=10)
> summary(out)

Call:
svm(formula = y ~ ., data = dat, kernel = "linear", cost = 10)


Parameters:
   SVM-Type:  C-classification
 SVM-Kernel:  linear
       cost:  10

Number of Support Vectors:  58

 ( 20 20 11 7 )


Number of Classes:  4

Levels:
 1 2 3 4
```

9.5. SVMs with More Than 2 Cases

Gene Expression Classification in R

Test Performance

```
> dat.te=data.frame(x=Khan$xtest , y=as.factor(Khan$ytest))
> pred.te=predict(out, newdata=dat.te)
> table(pred.te, dat.te$y)

pred.te 1 2 3 4
      1 3 0 0 0
      2 0 6 2 0
      3 0 0 4 0
      4 0 0 0 5
> sum(pred.te==dat.te$y)/20
[1] 0.9
```

9.5. SVMs with More Than 2 Cases

Gene Expression Classification in R

SVM easily overfits in high-dimensional case

```
> ##################################################### SVM
> svmfit.2=svm(y~., data=dat, kernel="radial",gamma=10,cost=10)
> dat.te=data.frame(x=Khan$xtest , y=as.factor(Khan$ytest))
> pred.te=predict(svmfit.2, newdata=dat.te)
> pred.tr=predict(svmfit.2,newdata=dat)
> table(pred.te, dat.te$y)

pred.te 1 2 3 4
      1 0 0 0 0
      2 3 6 6 5
      3 0 0 0 0
      4 0 0 0 0
> sum(pred.tr==dat$y)/63      ##### train error
[1] 1
> sum(pred.te==dat.te$y)/20   ##### test error
[1] 0.3
```

In this high-dimensional case, it is often better to apply support vector classifier.

9.6. Relation to Logistic Regression

Recall the "Loss + Penalty" formula

Minimize, for f in certain space,

$$\sum_{i=1}^{n} L(y_i, f(\mathbf{x}_i)) + \lambda P(f)$$

Ridge regression: for linear f,

$$\sum_{i=1}^{n} (y_i - f(\mathbf{x}_i))^2 + \lambda \sum_{j=1}^{p} \beta_j^2$$

Lasso regression: for linear f

$$\sum_{i=1}^{n} (y_i - f(\mathbf{x}_i))^2 + \lambda \sum_{j=1}^{p} |\beta_j|$$

9.6. Relation to Logistic Regression

Logistic regression for classification

Data: $(y_i, \mathbf{x}_i)$, $y_i = \pm 1$.

Model assumption

$$P(Y = 1|X) = 1/(1 + e^{-f(X)}); \quad P(Y = -1|X) = 1/(1 + e^{f(X)})$$

that is $P(Y = y|X) = 1/(1 + e^{-yf(X)})$

The negative of logistic likelihood (a.k.a. binomial deviance, cross-entropy)

$$\sum_{i=1}^{n} \log(1 + e^{-y_i f(\mathbf{x}_i)})$$

Note that in AdaBoost, exponential loss is used $\sum_{i=1}^{n} e^{-y_i f(\mathbf{x}_i)}$

9.6. Relation to Logistic Regression

Logistic regression with regularization

Logistic loss with ridge l_2 penalty

$$\sum_{i=1}^{n} \log(1 + e^{-y_i f(\mathbf{x}_i)}) + \lambda \sum_{j=1}^{p} \beta_j^2$$

Logistic loss with Lasso l_1 penalty:

$$\sum_{i=1}^{n} \log(1 + e^{-y_i f(\mathbf{x}_i)}) + \lambda \sum_{j=1}^{p} |\beta_j|$$

9.6. Relation to Logistic Regression

The SVM

Data: $(y_i, \mathbf{x}_i)$, $y_i = \pm 1$.

SVM is a result of "hige loss + ridge penalty":

$$\sum_{i=1}^{n} \max[0, 1 - y_i f(x_i)] + \lambda \sum_{j=1}^{p} \beta_j^2.$$

where $f(\mathbf{x}) = \beta_0 + \beta_1 x_1 + ... + \beta x_p$. Note that $\xi_i = \max[0, 1 - y_i f(x_i)] \geq 0$.

- the hinge loss function : $l(u) = (1 - u)_+$.
- the logistic loss function: $l(u) = \log(1 + e^{-u})$.
- the exponential loss function: $l(u) = e^{-u}$.

9.6. Relation to Logistic Regression

Figure: 9.12. The SVM and logistic regression loss functions are compared, as a function of $y_i(\beta_0 + \beta_1 x_{i1} + ...\beta_p x_{ip})$. When $y_i(\beta_0 + \beta_1 x_{i1} + ...\beta_p x_{ip})$ is greater than 1, then the SVM loss is zero, since this corresponds to an observation that is on the correct side of the margin. Overall, the two loss functions have quite similar behavior.

9.6. Relation to Logistic Regression

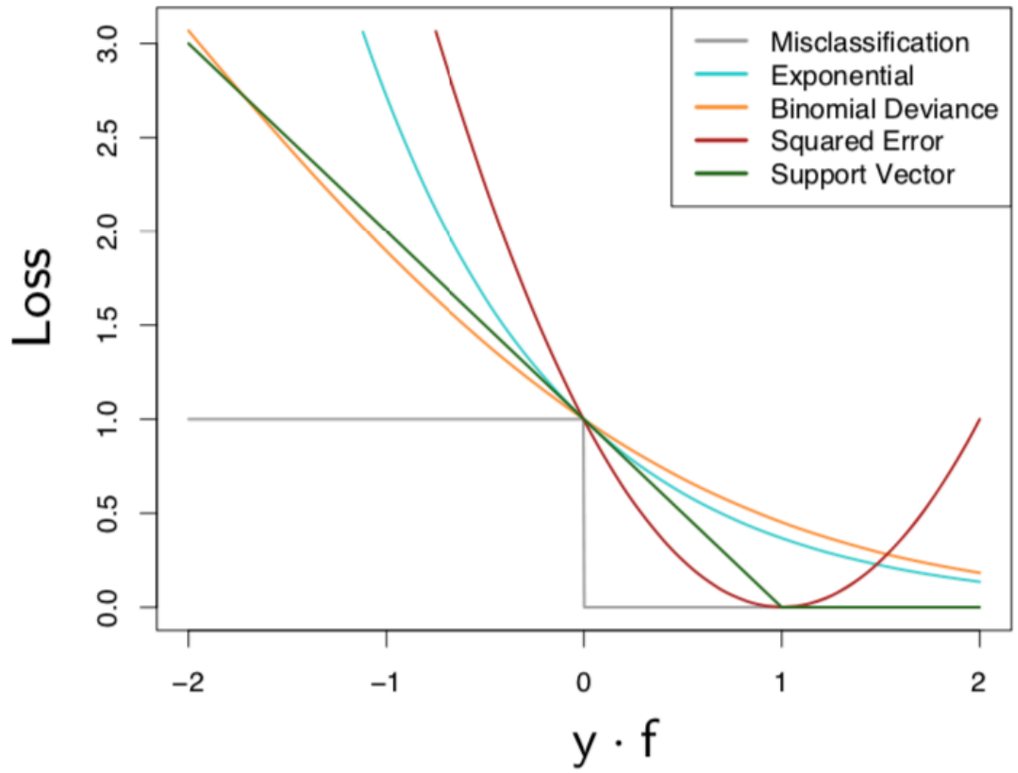


FIGURE 10.4. *Loss functions for two-class classification. The response is* $y = \pm 1$*; the prediction is* f*, with class prediction* $\text{sign}(f)$*. The losses are misclassification:* $I(\text{sign}(f) \neq y)$*; exponential:* $\exp(-yf)$*; binomial deviance:* $\log(1 + \exp(-2yf))$*; squared error:* $(y - f)^2$*; and support vector:* $(1 - yf)_+$ *(see Section 12.3). Each function has been scaled so that it passes through the point* $(0, 1)$*.*